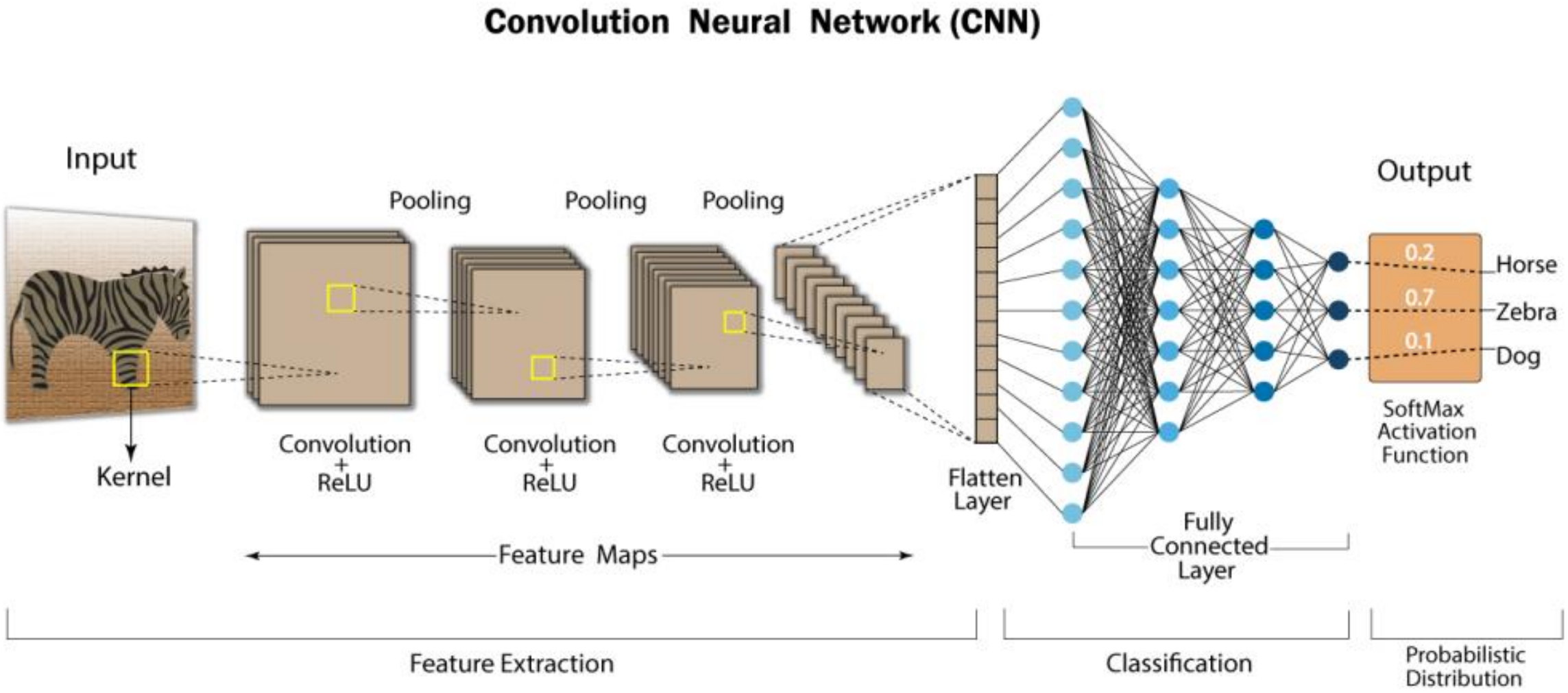
Unit 3

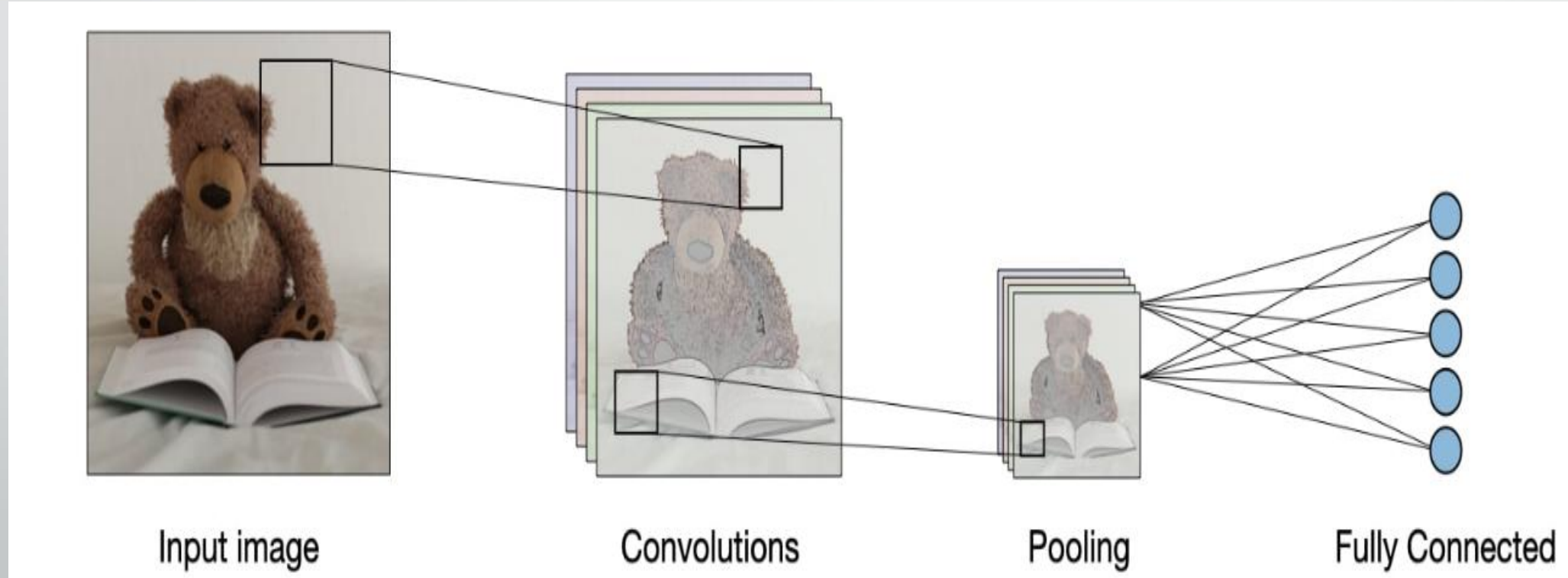
SDL

By Ms. Priyanka- Assistant Professor

Introduction to CNN

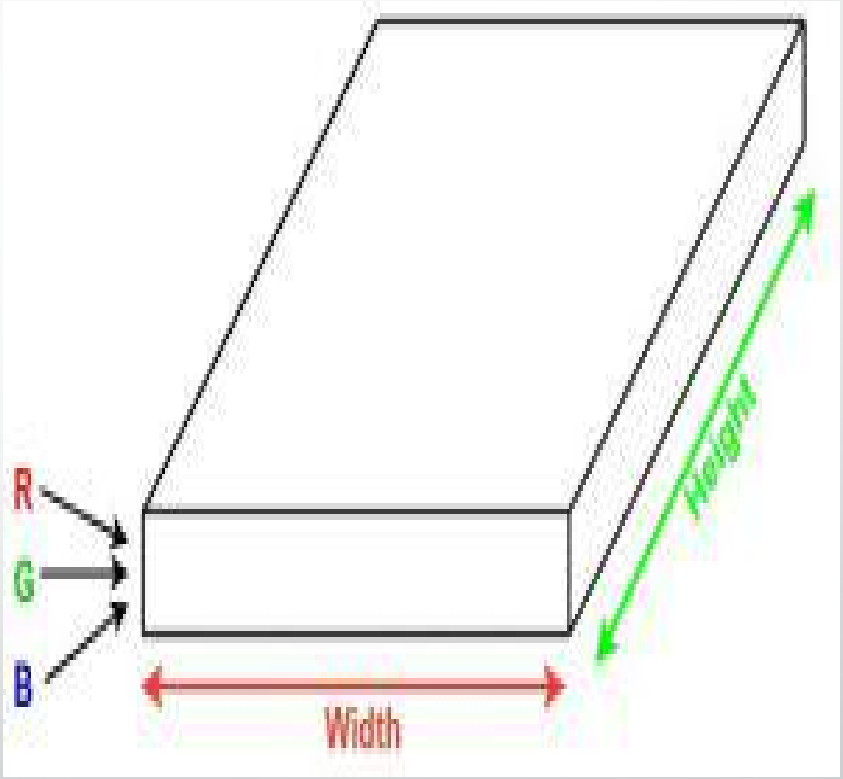
- **Convolutional Neural Network (CNN)** is an advanced version of **artificial neural networks (ANNs)**, primarily designed to extract features from grid-like matrix datasets. This is particularly useful for visual datasets such as images or videos, where data patterns play a crucial role. CNNs are widely used in **computer vision** applications due to their effectiveness in processing visual data.
- CNNs consist of multiple layers like the input layer, Convolutional layer, pooling layer, and fully connected layers.


Convolution Neural Network (CNN)
Input
Pooling
Pooling
Pooling
Output
0.2
Horse
0.7
Zebra
0.1
Dog
SoftMax
Activation
Function
Kernel
Convolution
+
ReLU
Convolution
+
ReLU
Convolution
+
ReLU
Flatten
Layer
Feature Maps
Fully
Connected
Layer
Feature Extraction
Classification
Probabilistic
Distribution

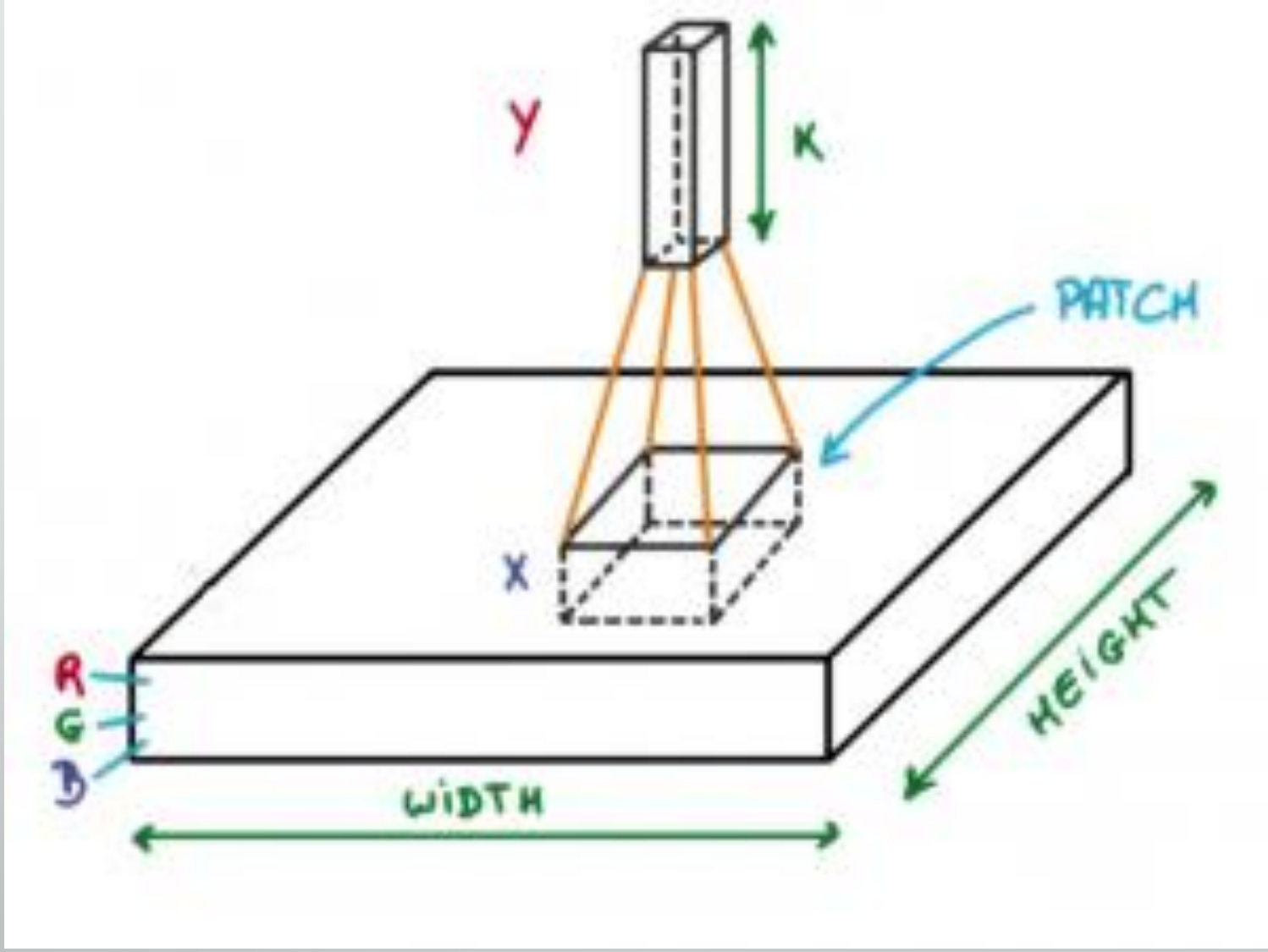

Input image
Convolutions
Pooling
Fully Connected

How Convolutional Layers Works?

- Convolution Neural Networks are neural networks that share their parameters.
- Imagine you have an image. It can be represented as a cuboid having its length, width (dimension of the image), and height (i.e the channel as images generally have red, green, and blue channels).



- Now imagine taking a small patch of this image and running a small neural network, called a filter or kernel on it, with say, K outputs and representing them vertically.
- Now slide that neural network across the whole image, as a result, we will get another image with different widths, heights, and depths. Instead of just R, G, and B channels now we have more channels but lesser width and height. This operation is called **Convolution**. If the patch size is the same as that of the image it will be a regular neural network. Because of this small patch, we have fewer weights.


Y
K
PATCH
X
R
G
B
WIDTH
HEIGHT

Mathematical Overview of Convolution

- Now let's talk about a bit of mathematics that is involved in the whole convolution process.
- Convolution layers consist of a set of learnable filters (or kernels) having small widths and heights and the same depth as that of input volume (3 if the input layer is image input).
- For example, if we have to run convolution on an image with dimensions 34x34x3. The possible size of filters can be axax3, where 'a' can be anything like 3, 5, or 7 but smaller as compared to the image dimension.
- During the forward pass, we slide each filter across the whole input volume step by step where each step is called **stride** (which can have a value of 2, 3, or even 4 for high-dimensional images) and compute the dot product between the kernel weights and patch from input volume.
- As we slide our filters we'll get a 2-D output for each filter and we'll stack them together as a result, we'll get output volume having a depth equal to the number of filters. The network will learn all the filters.

Layers Used to Build ConvNets

- A complete Convolution Neural Networks architecture is also known as covnets. A covnets is a sequence of layers, and every layer transforms one volume to another through a differentiable function.
- Let's take an example by running a covnets on of image of dimension 32 x 32 x 3.
- **Input Layers:** It's the layer in which we give input to our model. In CNN, Generally, the input will be an image or a sequence of images. This layer holds the raw input of the image with width 32, height 32, and depth 3.

- **<u>Convolutional Layers</u>:** This is the layer, which is used to extract the feature from the input dataset. It applies a set of learnable filters known as the kernels to the input images. The filters/kernels are smaller matrices usually 2x2, 3x3, or 5x5 shape. it slides over the input image data and computes the dot product between kernel weight and the corresponding input image patch. The output of this layer is referred as feature maps. Suppose we use a total of 12 filters for this layer we'll get an output volume of dimension 32 x 32 x 12.

7	2	3	3	8
4	5	3	8	4
3	3	2	8	4
2	8	7	2	7
5	4	4	5	4

*

1	0	-1
1	0	-1
1	0	-1

7x1+4x1+3x1+
2x0+5x0+3x0+
3x-1+3x-1+2x-1
= 6

=

6		

Convolution

Input (64, 64)

0.12 × -0.25	+ 0.12 × -0.22	+ 0.11 × 0.01	+
0.13 × -0.17	+ 0.14 × -0.26	+ 0.13 × -0.03	+
0.14 × -0.15	+ 0.14 × -0.02	+ 0.13 × -0.28	=

-0.18

Output (62, 62)

Hover over the matrices to change kernel position.

- **Activation Layer**: By adding an activation function to the output of the preceding layer, activation layers add nonlinearity to the network. it will apply an element-wise activation function to the output of the convolution layer. Some common activation functions are **RELU**: max(0, x), **Tanh**, **Leaky RELU**, etc. The volume remains unchanged hence output volume will have dimensions 32 x 32 x 12.

- **Pooling layer**: This layer is periodically inserted in the covnets and its main function is to reduce the size of volume which makes the computation fast reduces memory and also prevents overfitting. Two common types of pooling layers are **max pooling** and **average pooling**. If we use a max pool with 2 x 2 filters and stride 2, the resultant volume will be of dimension 16x16x12.

  Pooling (POOL) The pooling layer (POOL) is a downsampling operation, typically applied after a convolution layer, which does some spatial invariance. In particular, max and average pooling are special kinds of pooling where the maximum and average value is taken, respectively.

  Generally, noise, or any distortion is removed during pooling.

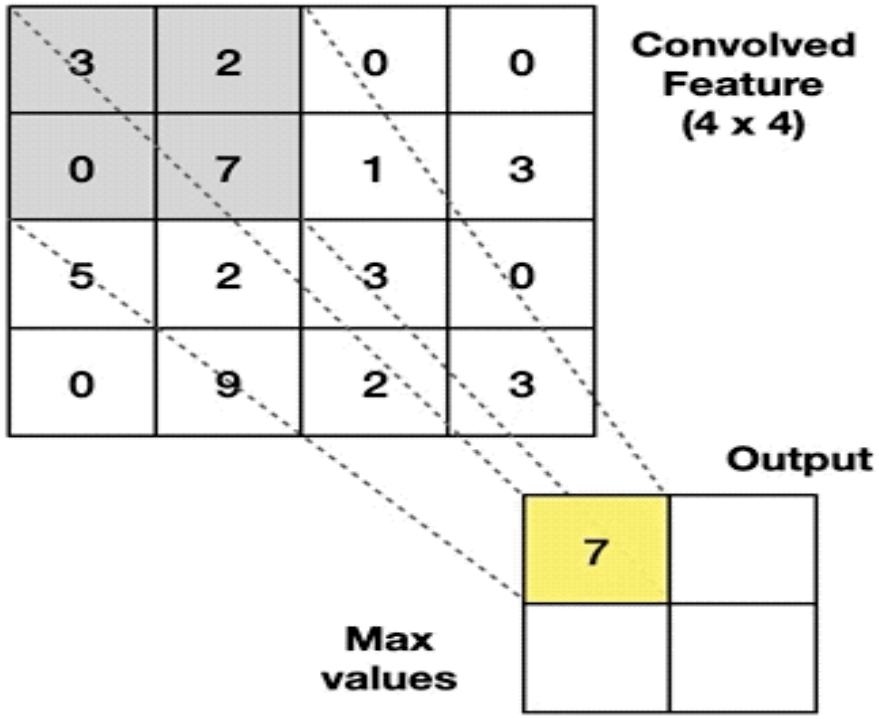
Max Pooling

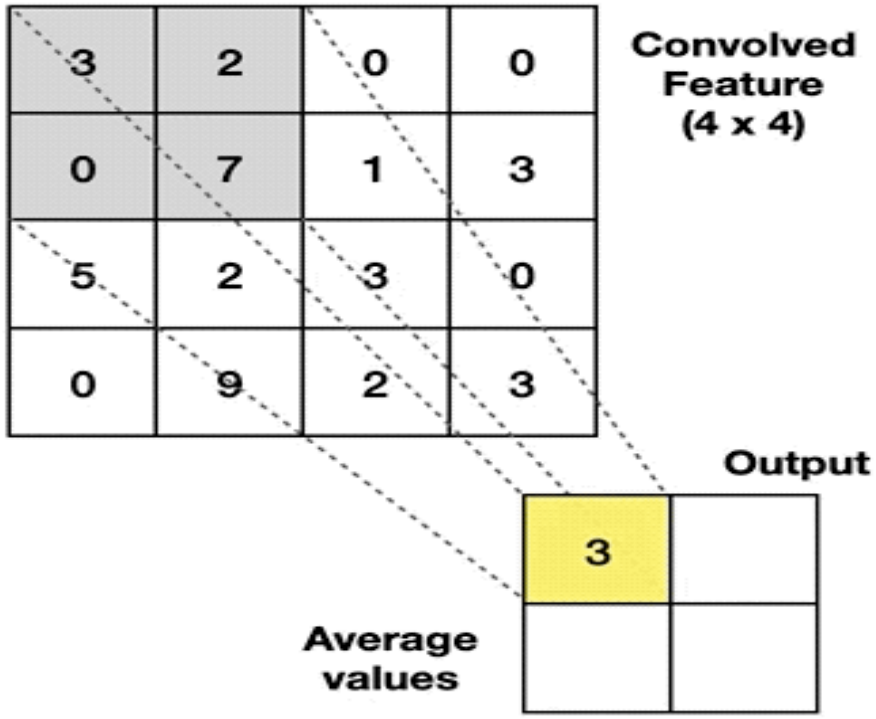
Take the **highest** value from the area covered by the kernel

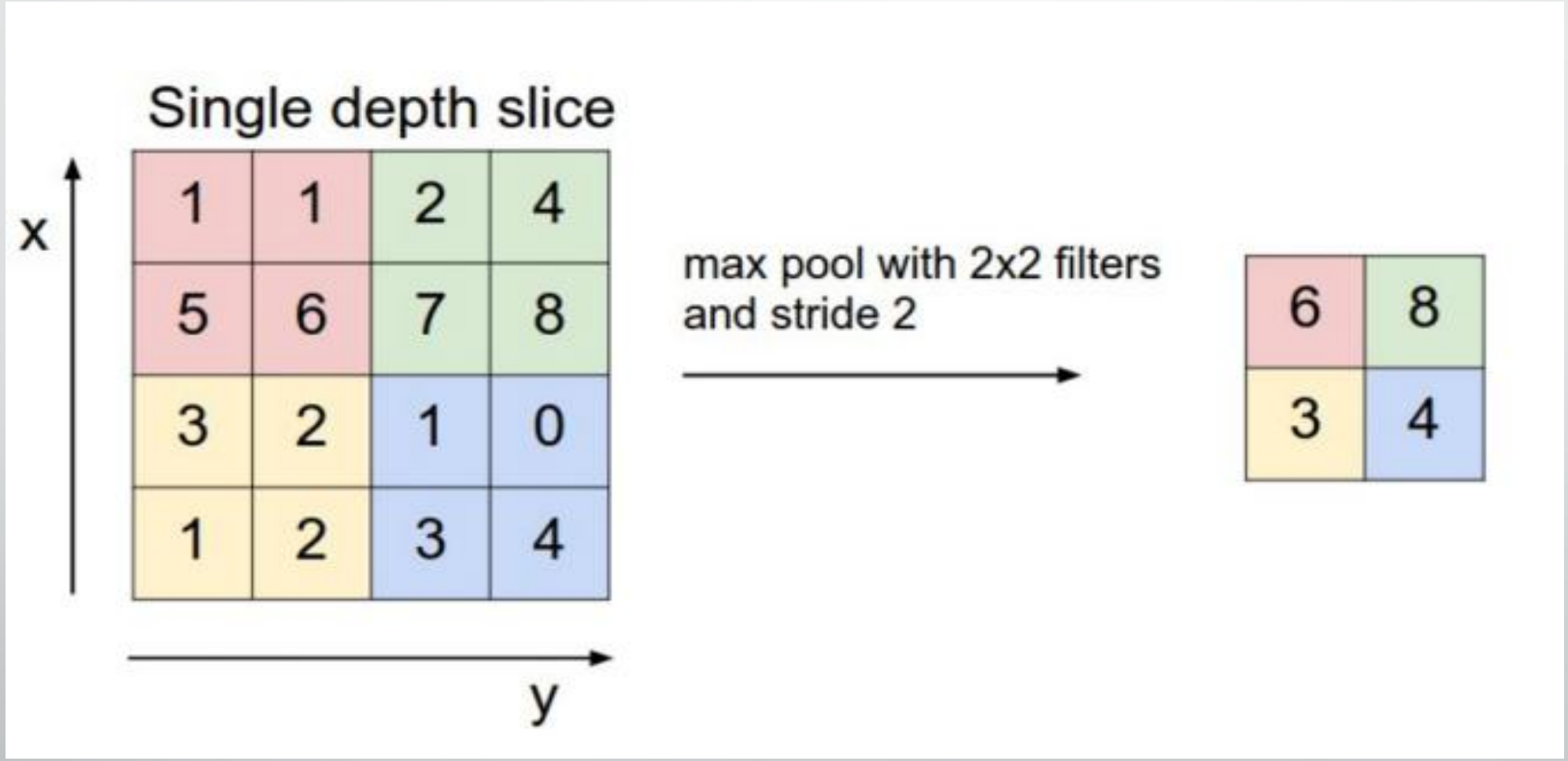
Average Pooling

Calculate the **average** value from the area covered by the kernel

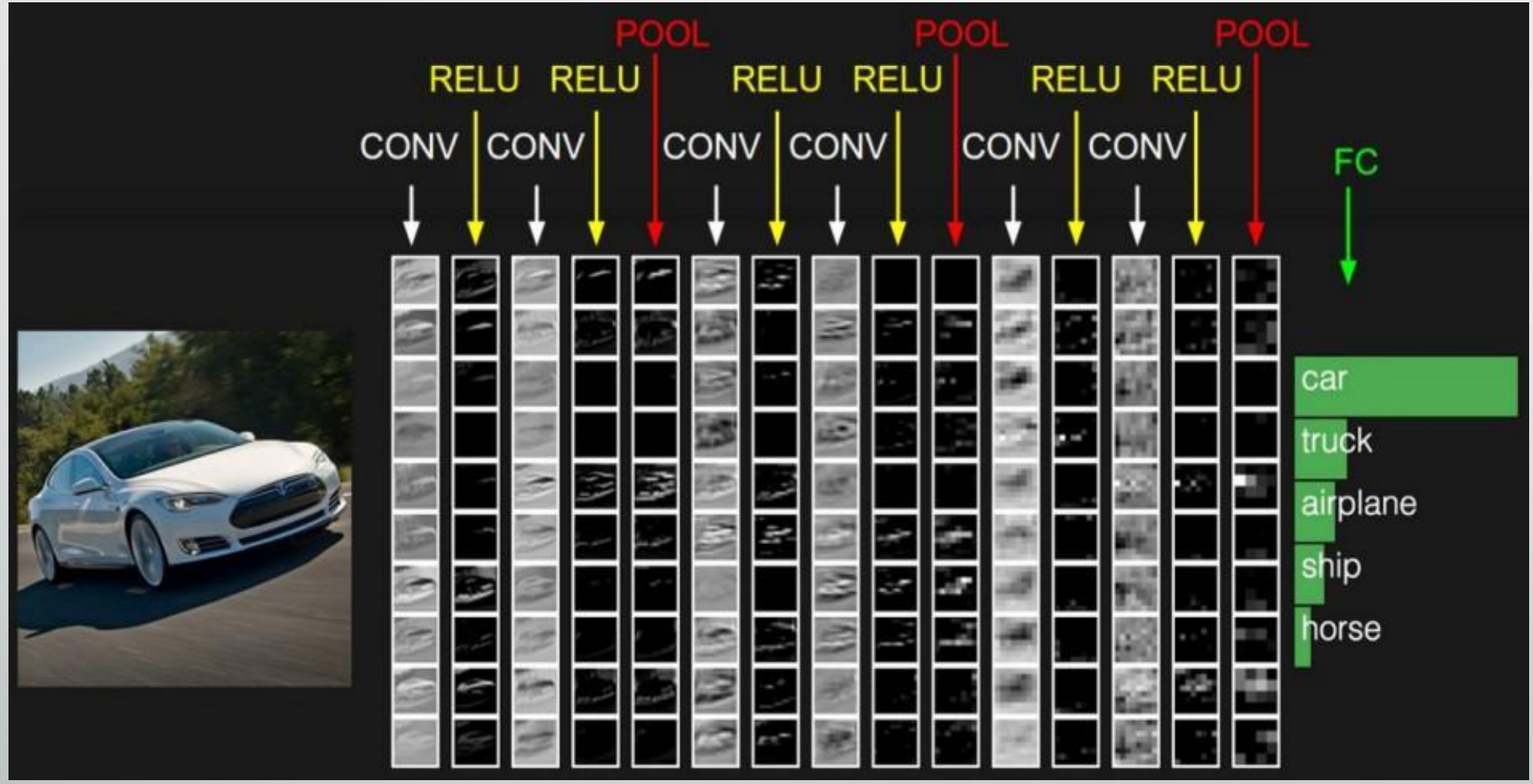
Example: Kernel of size 2 x 2; stride=(2,2)






Single depth slice
x
y
1 1 2 4
5 6 7 8
3 2 1 0
1 2 3 4
max pool with 2x2 filters
and stride 2
6 8
3 4

- **Flattening:** The resulting feature maps are flattened into a one-dimensional vector after the convolution and pooling layers so they can be passed into a completely linked layer for categorization or regression.
- **Fully Connected Layers:** It takes the input from the previous layer and computes the final classification or regression task.
- **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.


POOL
POOL
POOL
RELU
RELU
RELU
RELU
RELU
RELU
CONV
CONV
CONV
CONV
CONV
CONV
FC
car
truck
airplane
ship
horse

Padding

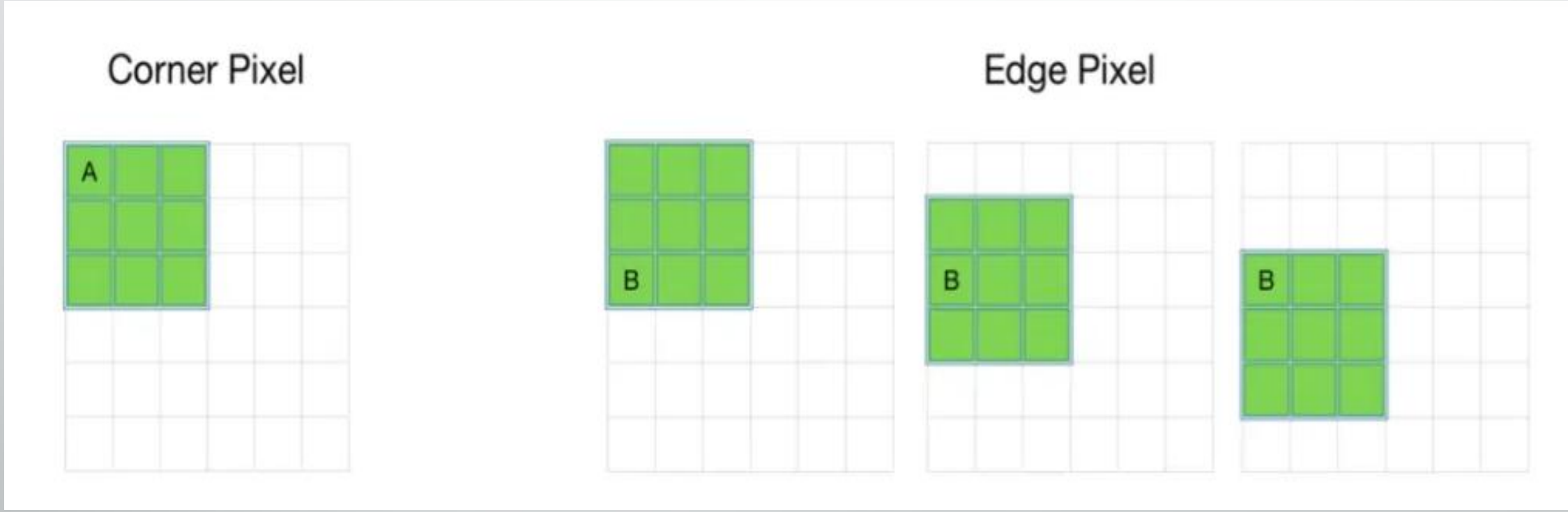
- padding is a technique used to preserve the spatial dimensions of the input image after convolution operations on a feature map. Padding involves adding extra pixels around the border of the input feature map before convolution.

- **This can be done in two ways:**
- **Valid Padding**: In the valid padding, no padding is added to the input feature map, and the output feature map is smaller than the input feature map. This is useful when we want to reduce the spatial dimensions of the feature maps.
- **Same Padding**: In the same padding, padding is added to the input feature map such that the size of the output feature map is the same as the input feature map. This is useful when we want to preserve the spatial dimensions of the feature maps.
- The number of pixels to be added for padding can be calculated based on the size of the kernel and the desired output of the feature map size. The most common padding value is zero-padding, which involves adding zeros to the borders of the input feature map.

Problem With Convolution Layers Without Padding

- For a grayscale (n x n) image and (f x f) filter/kernel, the dimensions of the image resulting from a convolution operation is **(n - f + 1) x (n - f + 1)**. For example, for an (8 x 8) image and (3 x 3) filter, the output resulting after the convolution operation would be of size (6 x 6). Thus, the image shrinks every time a convolution operation is performed. This places an upper limit to the number of times such an operation could be performed before the image reduces to nothing thereby precluding us from building deeper networks.

Also, the pixels on the corners and the edges are used much less than those in the middle.

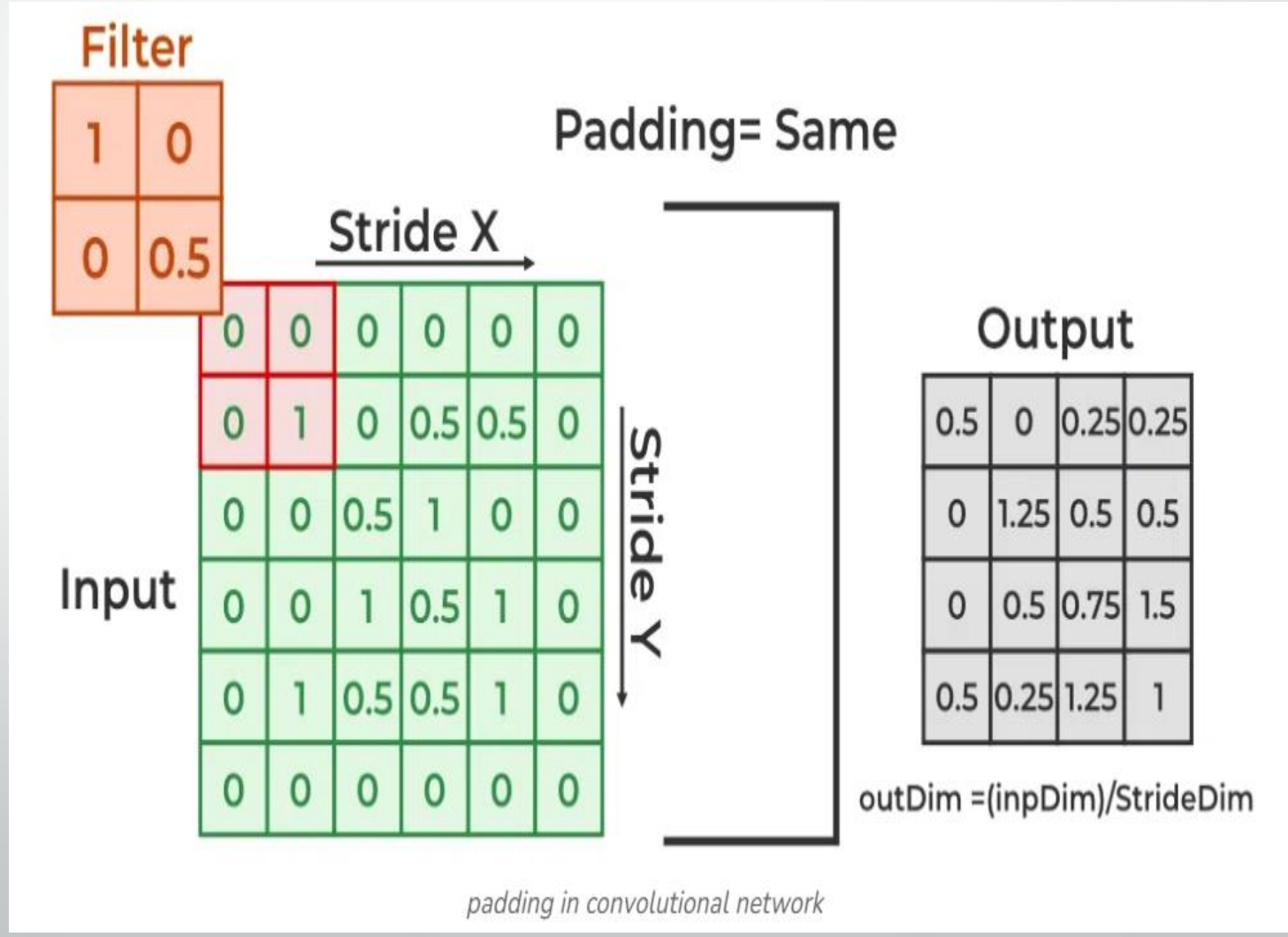



Middle Pixel
C
C
C
C
C
C
C
C
C

- Clearly, pixel A is touched in just one convolution operation and pixel B is touched in 3 convolution operations, while pixel C is touched in 9 convolution operations. In general, pixels in the middle are used more often than pixels on corners and edges. Consequently, the information on the borders of images is not preserved as well as the information in the middle.

Effect Of Padding On Input Images

- Padding is simply a process of adding layers of zeros to our input images so as to avoid the problems mentioned above through the following changes to the input image.



Padding prevents the shrinking of the input image.

__p__ = number of layers of zeros added to the border of the image,

then __(n x n)__ image ---> __(n + 2p) x (n + 2p)__ image after padding.

*__(n + 2p) x (n + 2p) * (f x f)__ -----> __outputs (n + 2p - f + 1) x (n + 2p - f + 1)__ images*

- For example, by adding one layer of padding to an (8 x 8) image and using a (3 x 3) filter we would get an (8 x 8) output after performing a convolution operation.
- This increases the contribution of the pixels at the border of the original image by bringing them into the middle of the padded image. Thus, information on the borders is preserved as well as the information in the middle of the image.

Types of Padding

- **Valid Padding:** It implies no padding at all. The input image is left in its valid/unaltered shape. So

$$(n \times n) * (f \times f) \longrightarrow (n - f + 1) \times (n - f + 1)$$

where, *nxn is the dimension of input image*

fxf is kernel size

n-f+1 is output image size

** represents a convolution operation.*

- **Same Padding:** In this case, we add 'p' padding layers such that the output image has the same dimensions as the input image. So,

*[(n + 2p) x (n + 2p) image] * [(f x f) filter] ---> [(n x n) image]*

which gives **p = (f - 1) / 2** (because n + 2p - f + 1 = n).

Object Detection

- Refer this link
- https://towardsdatascience.com/object-detection-with-convolutional-neural-networks-c9d729eedc18/
- https://www.analyticsvidhya.com/blog/2022/03/a-basic-introduction-to-object-detection/

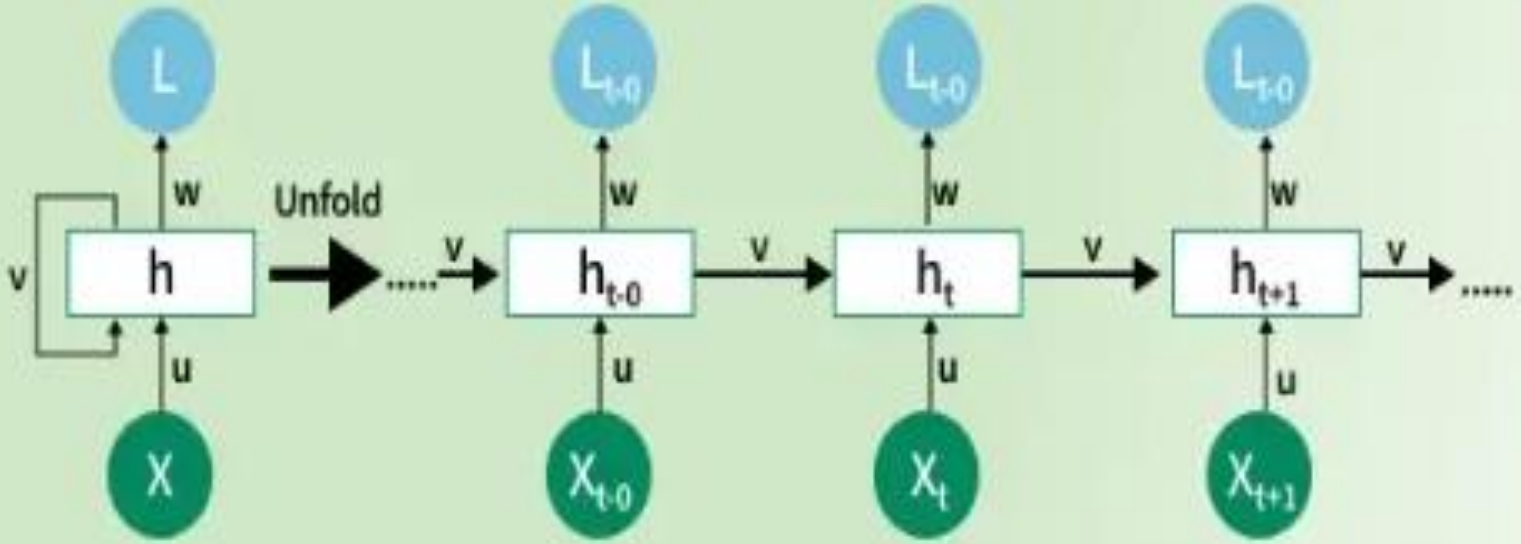
Image Segmentation

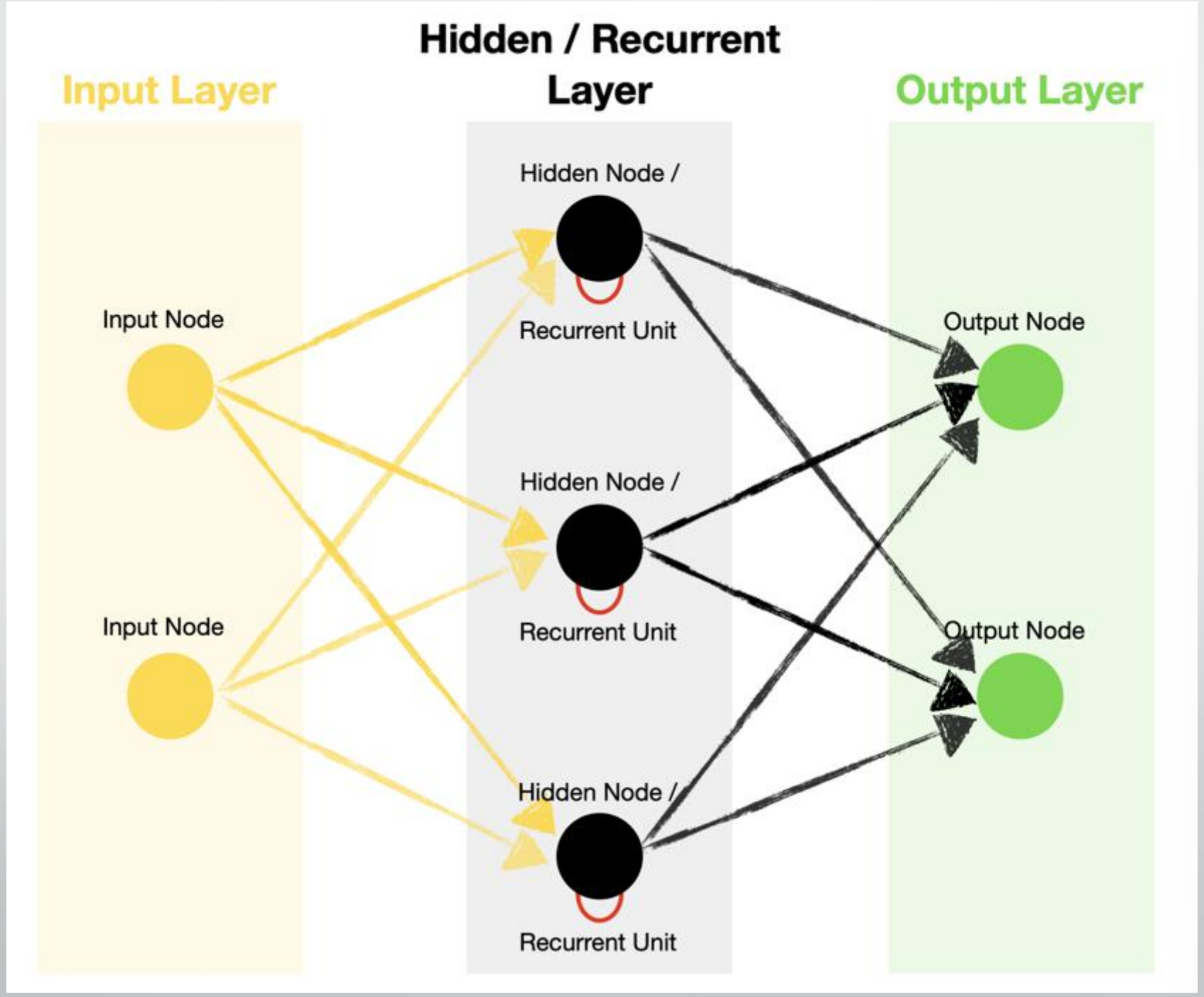
- Refer this link-
- https://www.analyticsvidhya.com/blog/2019/07/computer-vision-implementing-mask-r-cnn-image-segmentation/
- https://medium.com/@shameerayaseen21/u-net-advancing-image-segmentation-with-convolutional-neural-networks-1fd810f05d00

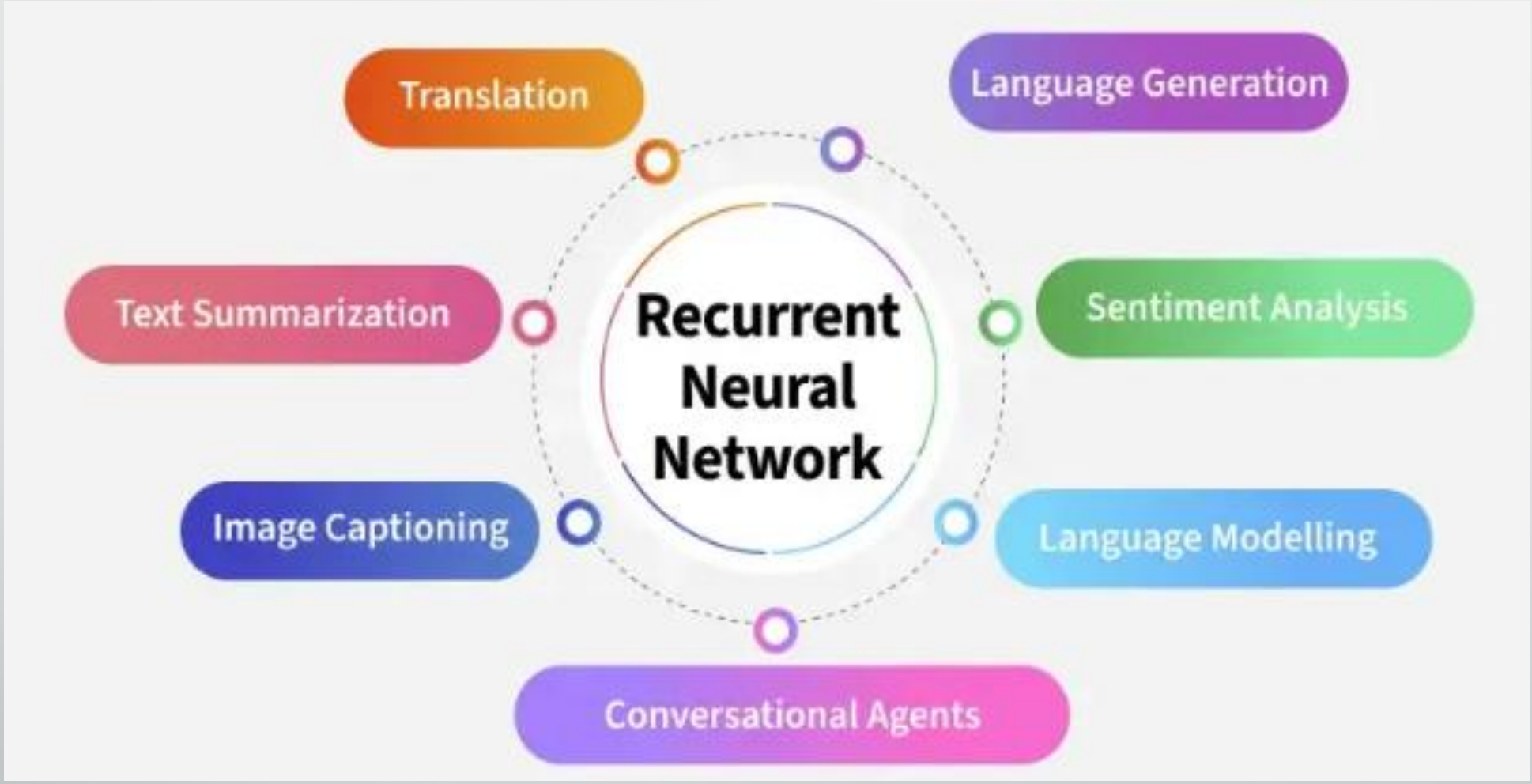
Introduction to Recurrent Neural Networks

- Recurrent Neural Networks (RNNs) differ from regular neural networks in how they process information. While standard neural networks pass information in one direction i.e. from input to output, RNNs feed information back into the network at each step.

Introduction to
Recurrent Neural Network



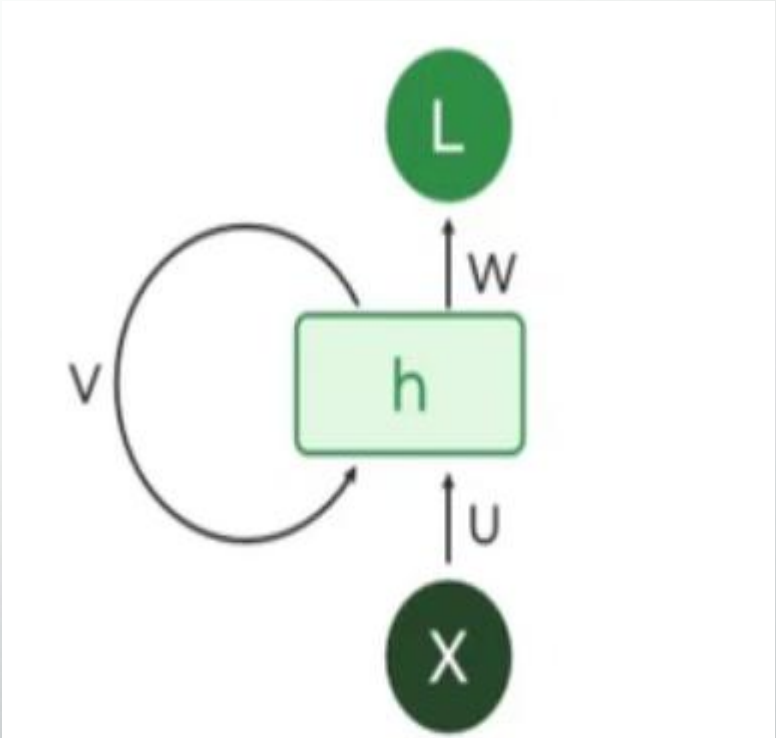

Hidden / Recurrent Layer
Input Layer
Output Layer
Hidden Node /
Input Node
Recurrent Unit
Output Node
Hidden Node /
Input Node
Recurrent Unit
Output Node
Hidden Node /
Recurrent Unit


Translation
Language Generation
Text Summarization
Recurrent Neural Network
Sentiment Analysis
Image Captioning
Language Modelling
Conversational Agents

- Imagine reading a sentence and you try to predict the next word, you don’t rely only on the current word but also remember the words that came before. RNNs work similarly by “remembering” past information and passing the output from one step as input to the next i.e it considers all the earlier words to choose the most likely next word. This memory of previous steps helps the network understand context and make better predictions.

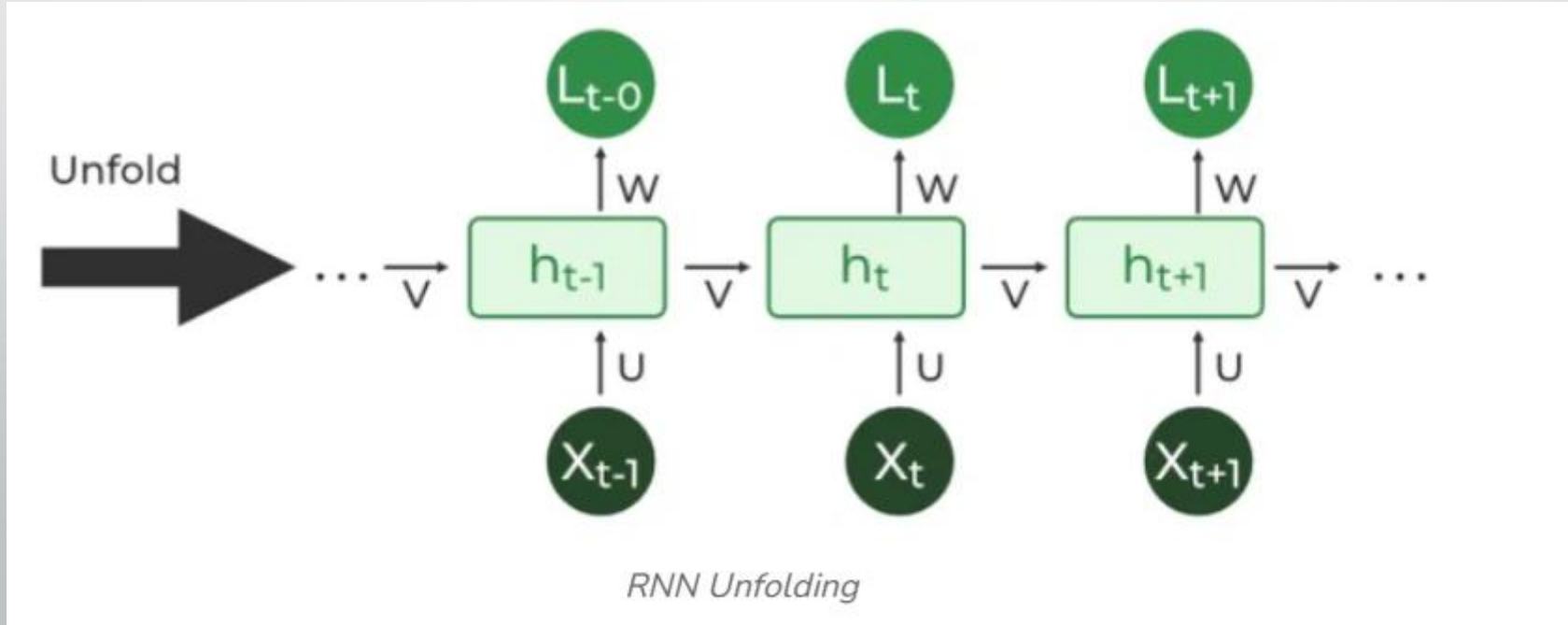
Key Components of RNNs

- There are mainly two components of RNNs that we will discuss.
- **1. Recurrent Neurons**
- The fundamental processing unit in RNN is a Recurrent Unit**.** They hold a hidden state that maintains information about previous inputs in a sequence. Recurrent units can "remember" information from prior steps by feeding back their hidden state, allowing them to capture dependencies across time.



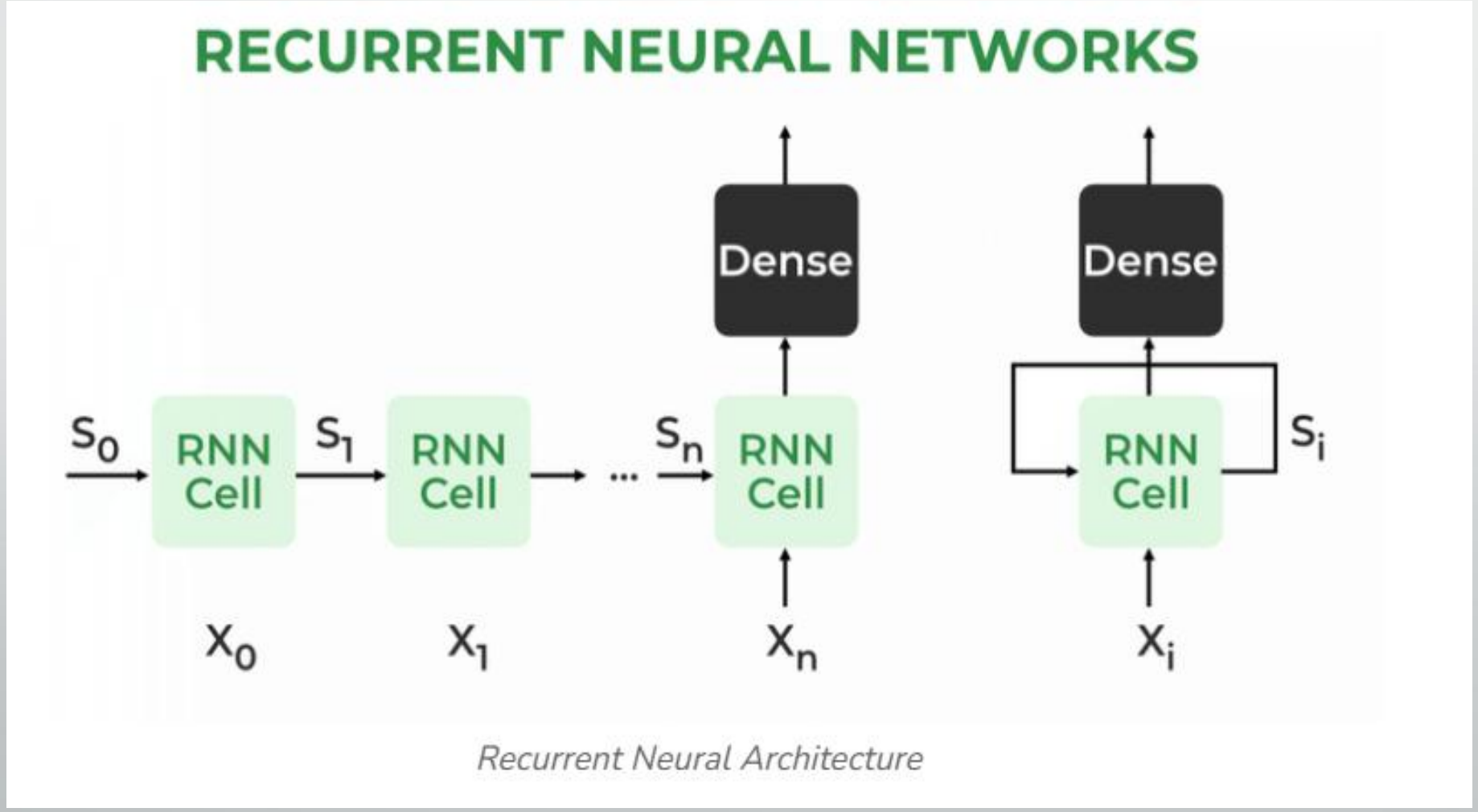
Recurrent Neuron

- **2. RNN Unfolding**
- RNN unfolding or unrolling is the process of expanding the recurrent structure over time steps. During unfolding each step of the sequence is represented as a separate layer in a series illustrating how information flows across each time step.
- This unrolling enables backpropagation through time (BPTT) a learning process where errors are propagated across time steps to adjust the network's weights enhancing the RNN's ability to learn dependencies within sequential data.



RNN Unfolding

Recurrent Neural Network Architecture



Recurrent Neural Architecture

- RNNs share similarities in input and output structures with other deep learning architectures but differ significantly in how information flows from input to output. Unlike traditional deep neural networks where each dense layer has distinct weight matrices. RNNs use shared weights across time steps, allowing them to remember information over sequences.
- In RNNs the hidden state H_i is calculated for every input X_i to retain sequential dependencies. The computations follow these core formulas:

1. Hidden State Calculation:

$$h = \sigma(U \cdot X + W \cdot h_{t-1} + B)$$

Here:

- h represents the current hidden state.
- U and W are weight matrices.
- B is the bias.

2. Output Calculation:

$$Y = O(V \cdot h + C)$$

The output Y is calculated by applying O an activation function to the weighted hidden state where V and C represent weights and bias.

3. Overall Function:

$$Y = f(X, h, W, U, V, B, C)$$

This function defines the entire RNN operation where the state matrix S holds each element s_i representing the network's state at each time step i.

How does RNN work?

- At each time step RNNs process units with a fixed activation function. These units have an internal hidden state that acts as memory that retains information from previous time steps. This memory allows the network to store past knowledge and adapt based on new inputs.
- **Updating the Hidden State in RNNs**
- The current hidden state h_t depends on the previous state h_{t-1} and the current input x_t and is calculated using the following relations:

1. State Update:

$$h_t = f(h_{t-1}, x_t)$$

where:

- h_t is the current state
- h_{t-1} is the previous state
- x_t is the input at the current time step

2. Activation Function Application:

$$h_t = \tanh(W_{hh} \cdot h_{t-1} + W_{xh} \cdot x_t)$$

Here, W_{hh} is the weight matrix for the recurrent neuron and W_{xh} is the weight matrix for the input neuron.

3. Output Calculation:

$$y_t = W_{hy} \cdot h_t$$

where y_t is the output and W_{hy} is the weight at the output layer.

These parameters are updated using backpropagation. However, since RNN works on sequential data here we use an updated backpropagation which is known as backpropagation through time.

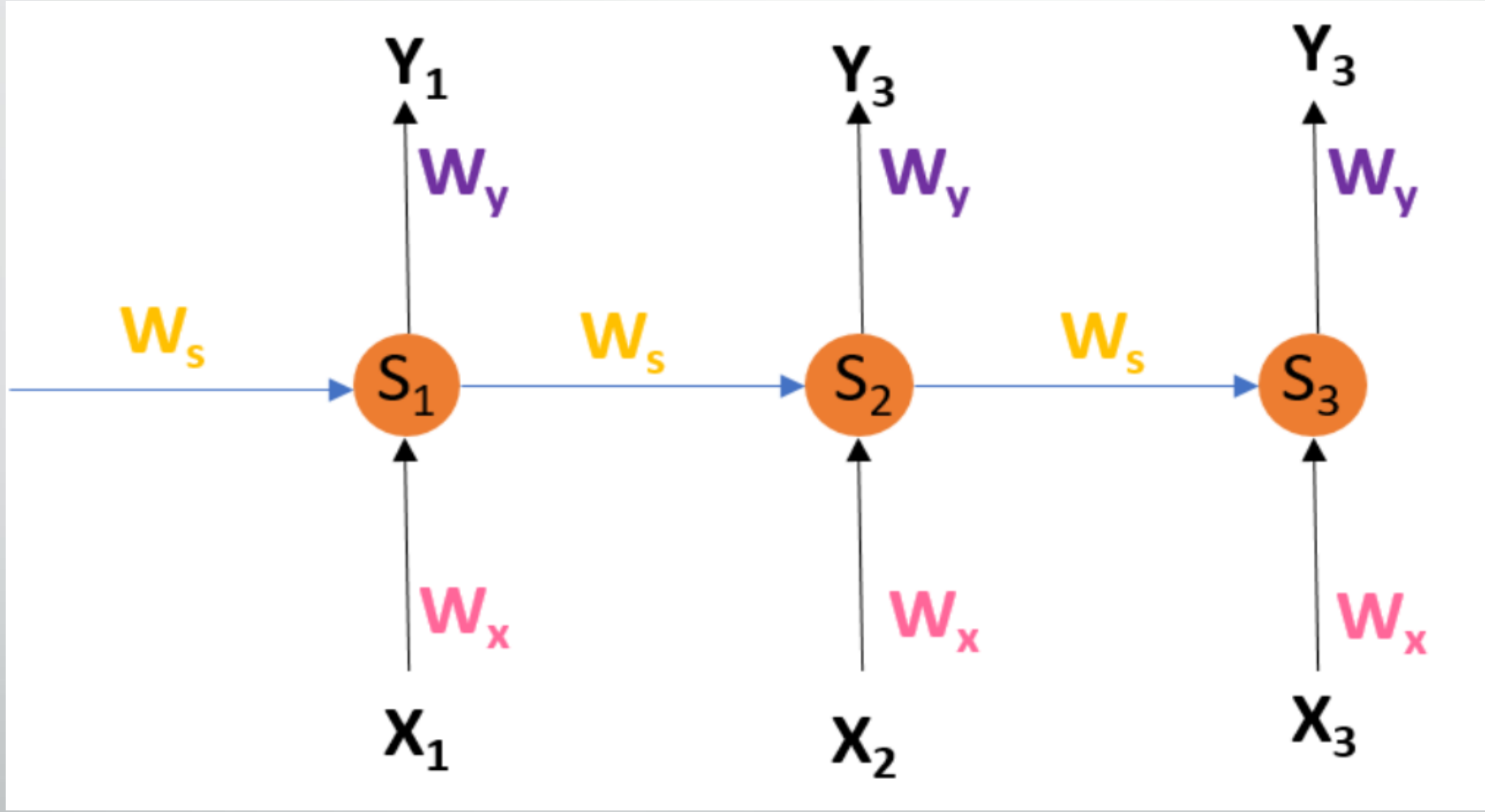
Backpropagation Through Time (BPTT) in RNNs

- Recurrent Neural Networks (RNNs) are designed to process sequential data. Unlike traditional neural networks, RNN outputs depend not only on the current input but also on previous inputs through a memory element. This memory allows RNNs to capture temporal dependencies in data such as time series or language.
- Training RNNs involves backpropagation where instead of updating weights based only on the current timestep t, we also consider all previous timesteps : $t-1, t-2, t-3, \ldots$.
- This method is called Backpropagation Through Time (BPTT) which extends traditional backpropagation to sequential data by unfolding the network over time and summing gradients across all relevant time steps. This method enables RNNs to learn complex temporal patterns.

RNN Architecture

- At each timestep t, the RNN maintains a hidden state S_t ,which acts as the network's memory summarizing information from previous inputs. The hidden state S_t updates by combining the current input X_t and the previous hidden state S_{t-1} ,applying an activation function to introduce non-linearity. Then the output Y_t is generated by transforming this hidden state.

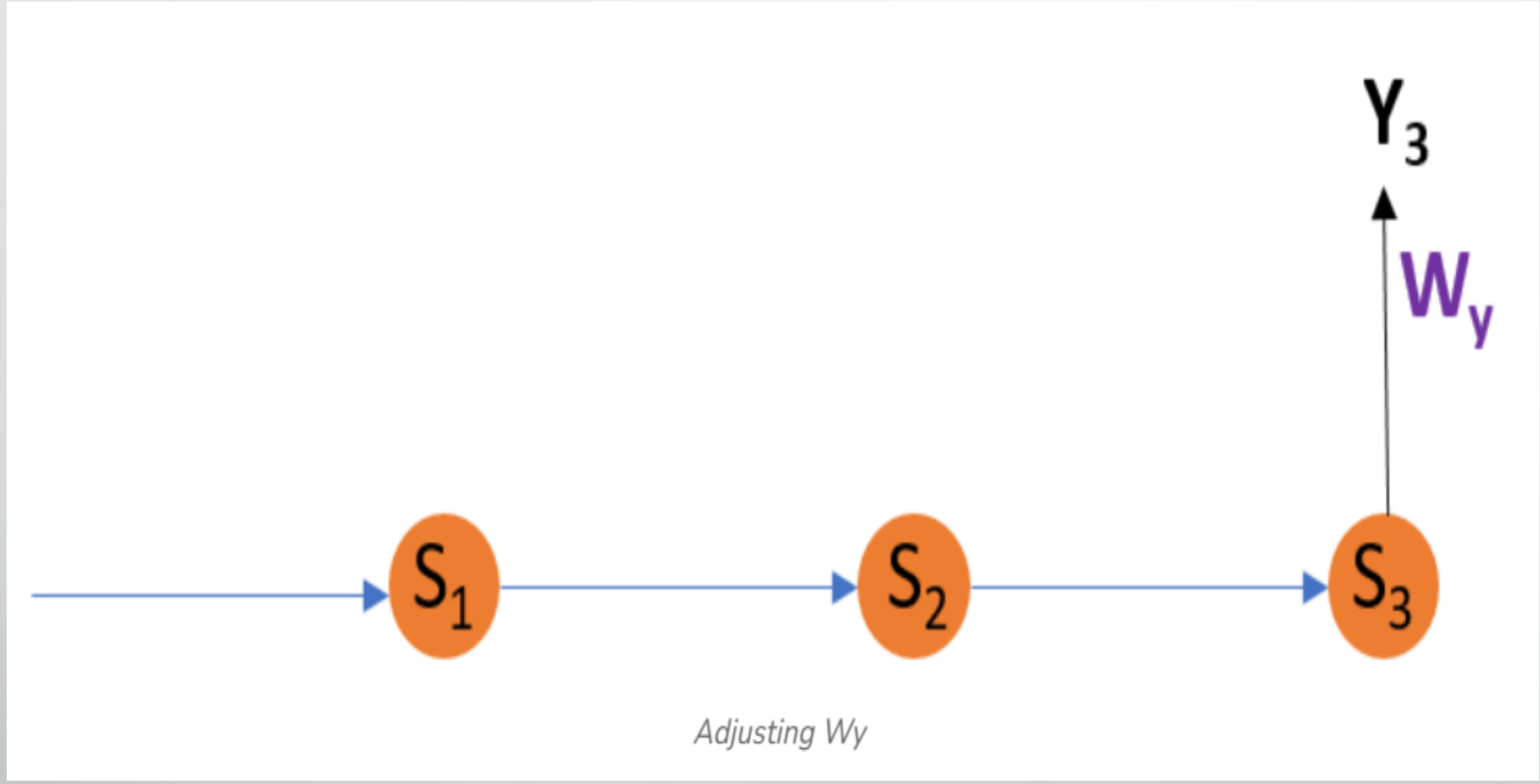
- $St=g1(Wx\,Xt+Ws\,St-1)$
- S_t represents the hidden state (memory) at time t.
- X_t is the input at time t.
- Y_t is the output at time t.
- W_s, W_x, W_y are weight matrices for hidden states, inputs and outputs, respectively.
- $Y_t = g_2(W_y S_t)$
- where g_1 and g_2 are activation functions.


Y1
Y3
Y3
Wy
Wy
Wy
Ws
Ws
Ws
S1
S2
S3
Wx
Wx
Wx
X1
X2
X3

- **Error Function at Time $t = 3$**
- To train the network, we measure how far the predicted output Y_t is from the desired output d_t using an error function. We use the squared error to measure the difference between the desired output d_t and actual output Y_t:
- $E_t = \left(d_t - Y_t\right)^2$
- At $t = 3$:
- $E_3 = \left(d_3 - Y_3\right)^2$
- This error quantifies the difference between the predicted output and the actual output at time 3.

- **Updating Weights Using BPTT**
- BPTT updates the weights W_y, W_s, W_x to minimize the error by computing gradients. Unlike standard backpropagation, BPTT unfolds the network across time steps, considering how errors at time t depend on all previous states.
- We want to adjust the weights W_y ,W_s and W_x to minimize the error E_3.
- **.1Adjusting Output Weight** W_y
- The output weight W_y affects the output directly at time 3. This means we calculate how the error changes as Y_3 changes, then how Y_3 changes with respect to W_y .Updating W_y is straightforward because it only influences the current output.

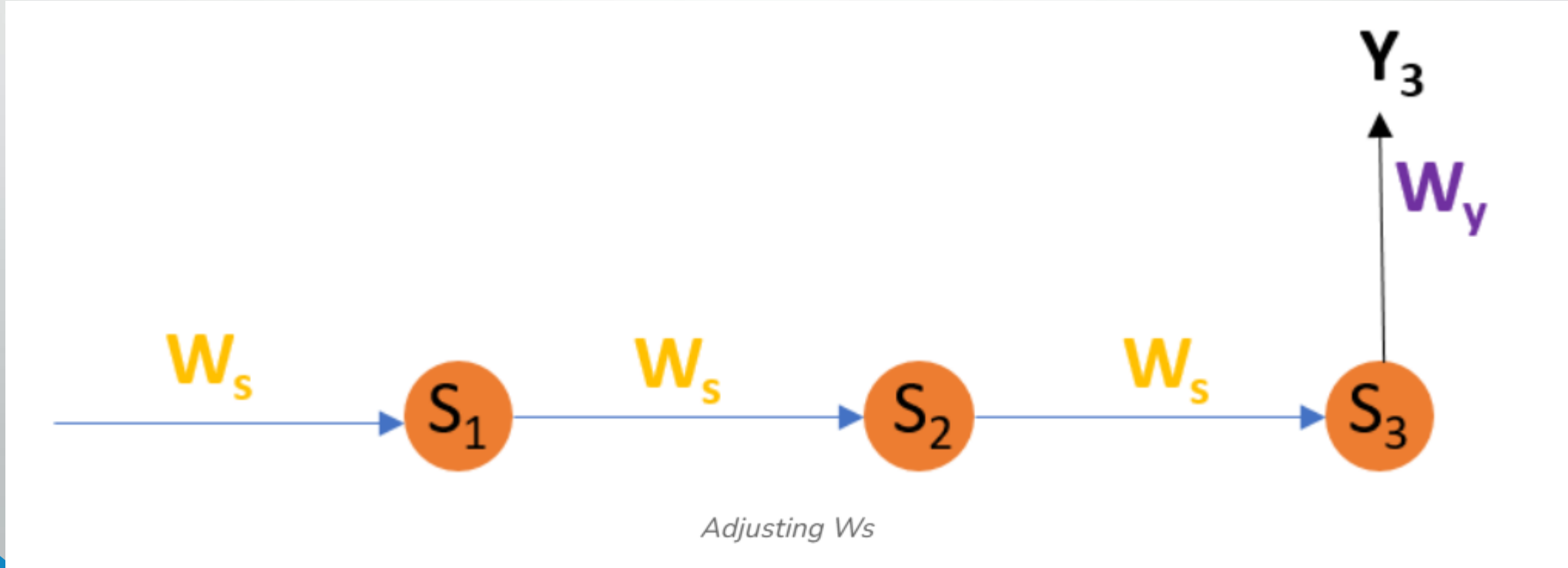
- Using the chain rule:
- $\frac{\partial E_3}{\partial W_y} = \frac{\partial E_3}{\partial Y_3} \times \frac{\partial Y_3}{\partial W_y}$
- E_3 depends on Y_3 ,so we differentiate E_3 w.r.t. Y_3.
- Y_3 depends on W_y ,so we differentiate Y_3 w.r.t. W_y.



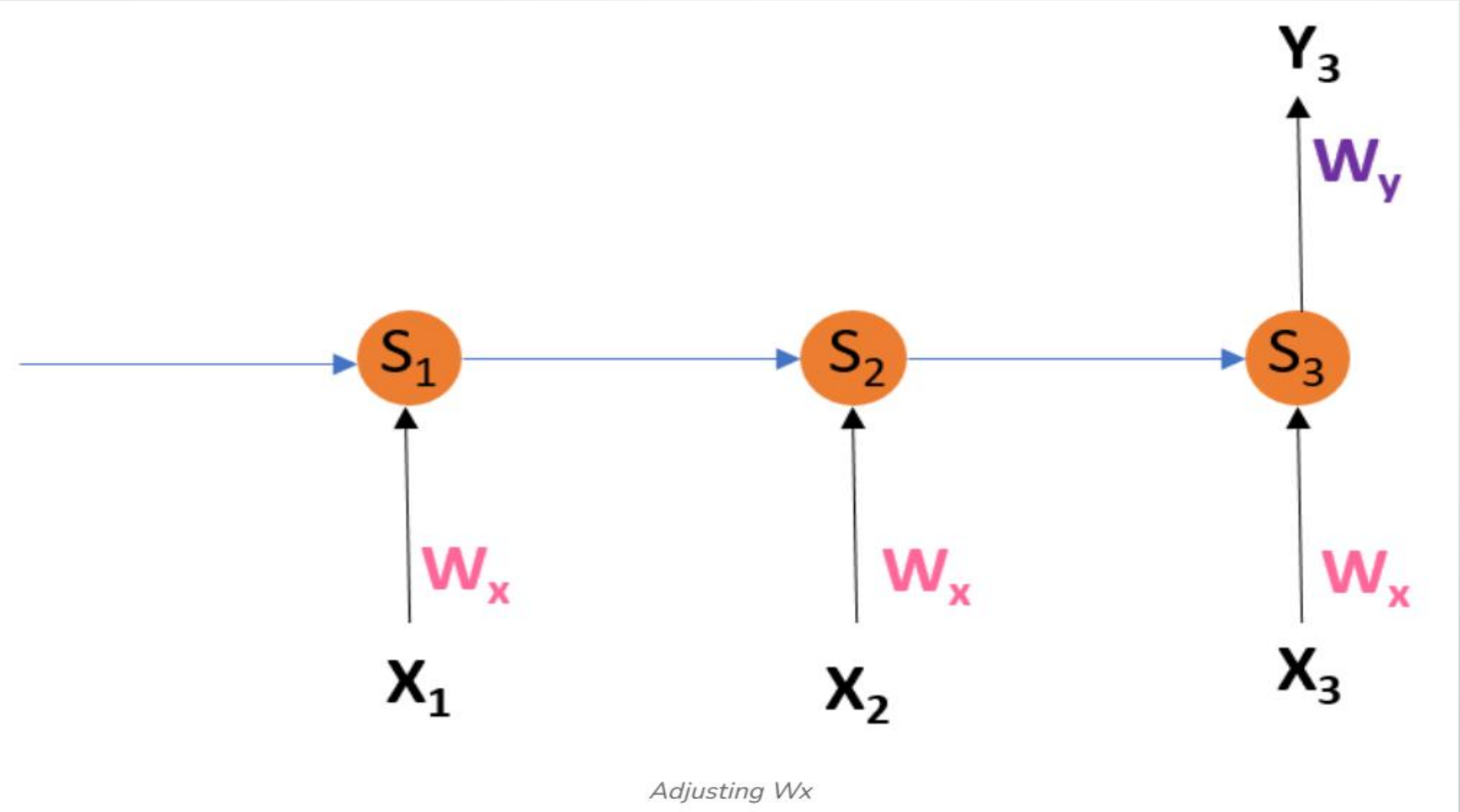
Adjusting Wy

- **2. Adjusting Hidden State Weight W_s**
- The hidden state weight W_s influences not just the current hidden state but all previous ones because each hidden state depends on the previous one. To update W_s ,we must consider how changes to W_s affect all hidden statesS_1, S_2, S_3 and consequently the output at time 3.
- The gradient for W_s considers all previous hidden states because each hidden state depends on the previous one:
- $$\frac{\partial E_3}{\partial W_s} = \sum_{i=1}^{3} \frac{\partial E_3}{\partial Y_3} \times \frac{\partial Y_3}{\partial S_i} \times \frac{\partial S_i}{\partial W_s}$$

- **Breaking down:**
- Start with the error gradient at output Y_3.
- Propagate gradients back through all hidden states S_3, S_2, S_1 since they affect Y_3.
- Each S_i depends on W_s ,so we differentiate accordingly.



- **3. Adjusting Input Weight W_x**
- Similar to W_s ,the input weight W_x affects all hidden states because the input at each timestep shapes the hidden state. The process considers how every input in the sequence impacts the hidden states leading to the output at time 3.
- $$\frac{\partial E_3}{\partial W_x} = \sum_{i=1}^{3} \frac{\partial E_3}{\partial Y_3} \times \frac{\partial Y_3}{\partial S_i} \times \frac{\partial S_i}{\partial W_x}$$
- The process is similar to W_s ,accounting for all previous hidden states because inputs at each timestep affect the hidden states.



Adjusting Wx

Advantages of Backpropagation Through Time (BPTT)

- **Captures Temporal Dependencies:** BPTT allows RNNs to learn relationships across time steps, crucial for sequential data like speech, text and time series.
- **Unfolding over Time:** By considering all previous states during training, BPTT helps the model understand how past inputs influence future outputs.
- **Foundation for Modern RNNs:** BPTT forms the basis for training advanced architectures such as LSTMs and GRUs, enabling effective learning of long sequences.
- **Flexible for Variable Length Sequences**: It can handle input sequences of varying lengths, adapting gradient calculations accordingly.

Limitations of BPTT

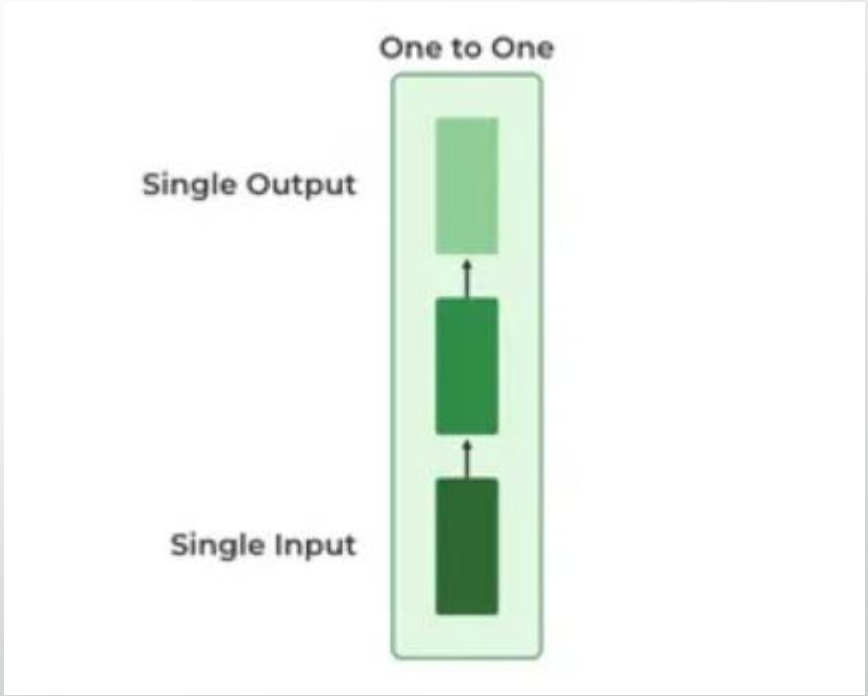
- **Vanishing Gradient Problem:** When backpropagating over many time steps, gradients tend to shrink exponentially, making early time steps contribute very little to weight updates. This causes the network to "forget" long-term dependencies.
- **Exploding Gradient Problem:** Gradients may also grow uncontrollably large, causing unstable updates and making training difficult.

Solutions

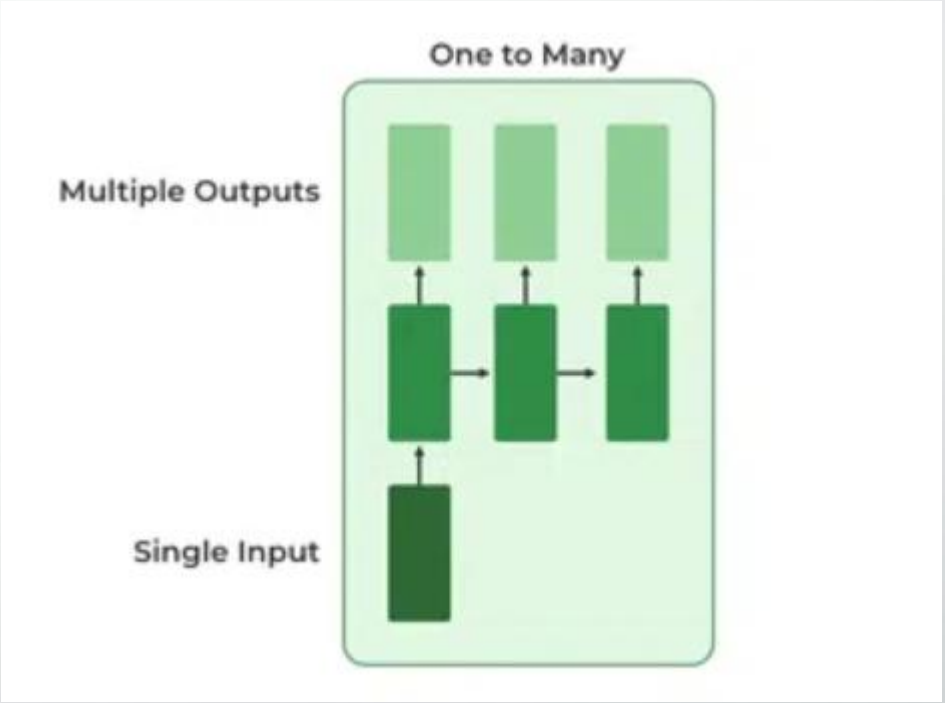
- **Long Short-Term Memory (LSTM):** Special RNN cells designed to maintain information over longer sequences and mitigate vanishing gradients.
- **Gradient Clipping:** Limits the magnitude of gradients during backpropagation to prevent explosion by normalizing them when exceeding a threshold.

TYPES OF RNN

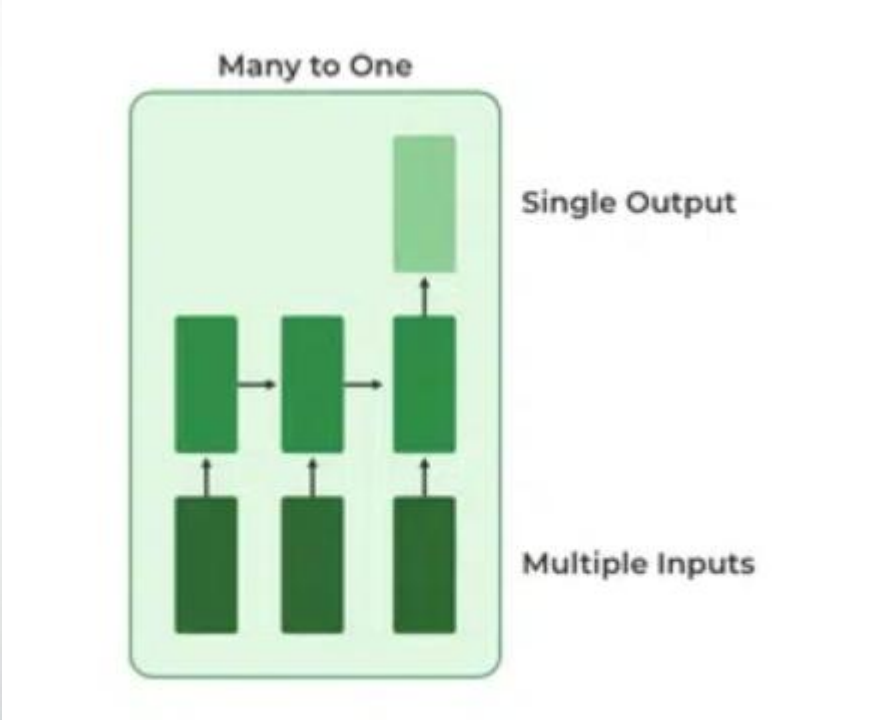
- **1. One-to-One RNN**
- This is the simplest type of neural network architecture where there is a single input and a single output. It is used for straightforward classification tasks such as binary classification where no sequential data is involved.



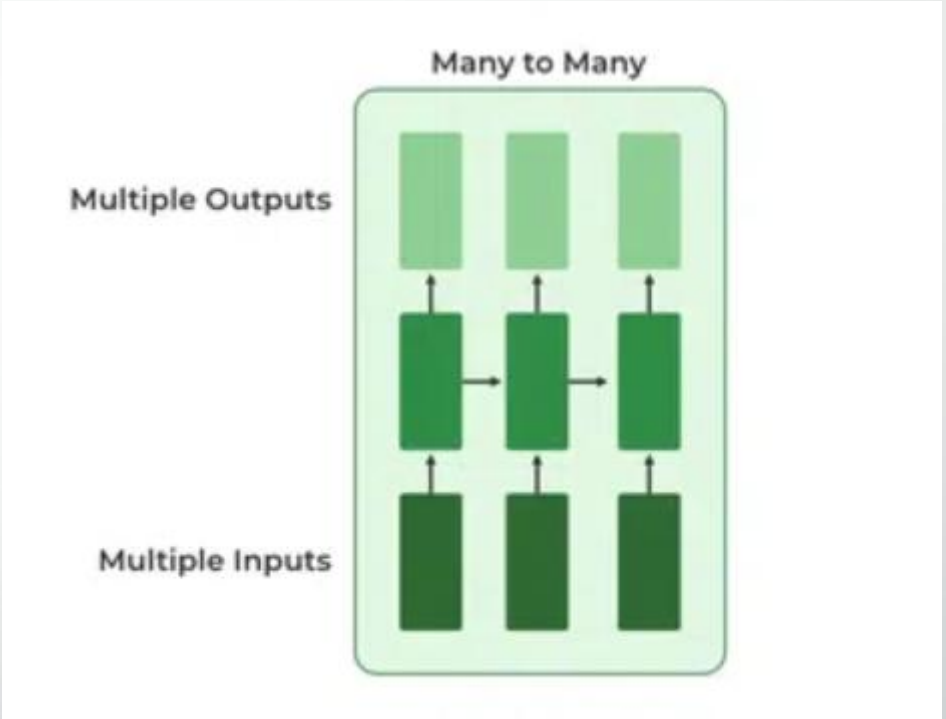
- **2. One-to-Many RNN**
- In a One-to-Many RNN the network processes a single input to produce multiple outputs over time. This is useful in tasks where one input triggers a sequence of predictions (outputs). For example, in image captioning a single image can be used as input to generate a sequence of words as a caption.



- **3. Many-to-One RNN**
- The Many-to-One RNN receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis the model receives a sequence of words (like a sentence) and produces a single output like positive, negative or neutral.



- **4. Many-to-Many RNN**
- The Many-to-Many RNN type processes a sequence of inputs and generates a sequence of outputs. In language translation task a sequence of words in one language is given as input and a corresponding sequence in another language is generated as output.



Variants of Recurrent Neural Networks (RNNs)

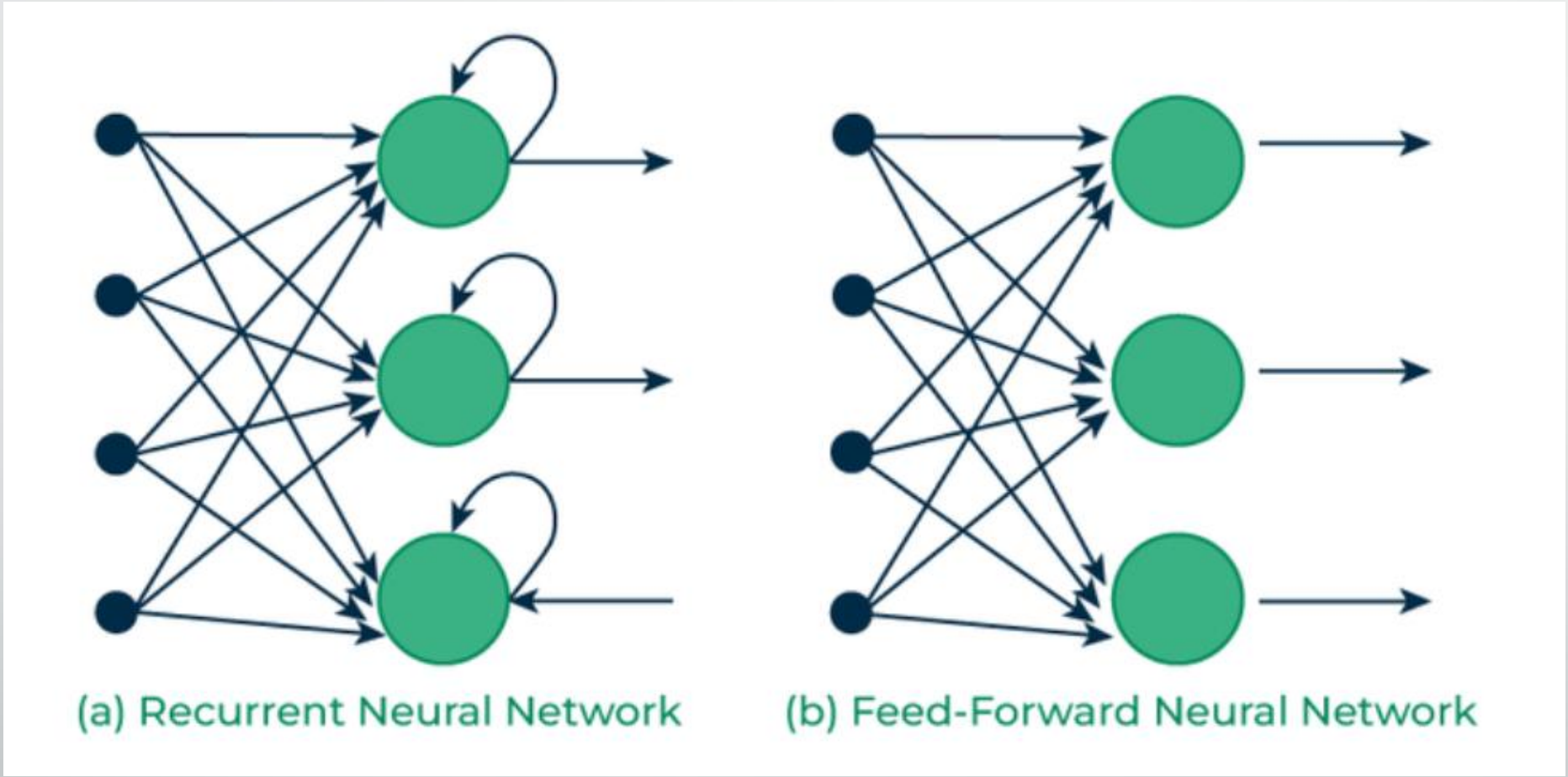
- There are several variations of RNNs, each designed to address specific challenges or optimize for certain tasks:
- **1. Vanilla RNN**
- This simplest form of RNN consists of a single hidden layer where weights are shared across time steps. Vanilla RNNs are suitable for learning short-term dependencies but are limited by the vanishing gradient problem, which hampers long-sequence learning.

- **2. Bidirectional RNNs**
- Bidirectional RNNs process inputs in both forward and backward directions, capturing both past and future context for each time step. This architecture is ideal for tasks where the entire sequence is available, such as named entity recognition and question answering.
- **3. Long Short-Term Memory Networks (LSTMs)**
- Long Short-Term Memory Networks (LSTMs) introduce a memory mechanism to overcome the vanishing gradient problem. Each LSTM cell has three gates:
- **Input Gate**: Controls how much new information should be added to the cell state.
- **Forget Gate**: Decides what past information should be discarded.
- **Output Gate**: Regulates what information should be output at the current step. This selective memory enables LSTMs to handle long-term dependencies, making them ideal for tasks where earlier context is critical.

- **4. Gated Recurrent Units (GRUs)**
- Gated Recurrent Units (GRUs) simplify LSTMs by combining the input and forget gates into a single update gate and streamlining the output mechanism. This design is computationally efficient, often performing similarly to LSTMs and is useful in tasks where simplicity and faster training are beneficial.

How RNN Differs from Feedforward Neural Networks?

- Feedforward Neural Networks (FNNs) process data in one direction from input to output without retaining information from previous inputs. This makes them suitable for tasks with independent inputs like image classification. However, FNNs struggle with sequential data since they lack memory.
- Recurrent Neural Networks (RNNs) solve this by incorporating loops that allow information from previous steps to be fed back into the network. This feedback enables RNNs to remember prior inputs making them ideal for tasks where context is important.


(a) Recurrent Neural Network
(b) Feed-Forward Neural Network

Advantages

- **Sequential Memory**: RNNs retain information from previous inputs making them ideal for time-series predictions where past data is crucial.
- **Enhanced Pixel Neighborhoods**: RNNs can be combined with convolutional layers to capture extended pixel neighborhoods improving performance in image and video data processing.

Limitations

- While RNNs excel at handling sequential data they face two main training challenges i.e vanishing gradient and exploding gradient problem:
- **Vanishing Gradient**: During backpropagation gradients diminish as they pass through each time step leading to minimal weight updates. This limits the RNN's ability to learn long-term dependencies which is crucial for tasks like language translation.
- **Exploding Gradient**: Sometimes gradients grow uncontrollably causing excessively large weight updates that de-stabilize training.
- **Slow training time:** An RNN processes data sequentially, which limits its ability to process a large number of texts efficiently. For example, an RNN model can analyze a buyer's sentiment from a couple of sentences. However, it requires massive computing power, memory space, and time to summarize a page of an essay.

- These challenges can hinder the performance of standard RNNs on complex, long-sequence tasks.

How do transformers overcome the limitations of recurrent neural networks?

- Transformers are deep learning models that use self-attention mechanisms in an encoder-decoder feed-forward neural network. They can process sequential data the same way that RNNs do.
- **Self-attention**
- Transformers don’t use hidden states to capture the interdependencies of data sequences. Instead, they use a self-attention head to process data sequences in parallel. This enables transformers to train and process longer sequences in less time than an RNN does. With the self-attention mechanism, transformers overcome the memory limitations and sequence interdependencies that RNNs face. Transformers can process data sequences in parallel and use positional encoding to remember how each input relates to others.

- **Parallelism**
- Transformers solve the gradient issues that RNNs face by enabling parallelism during training. By processing all input sequences simultaneously, a transformer isn't subjected to backpropagation restrictions because gradients can flow freely to all weights. They are also optimized for parallel computing, which graphic processing units (GPUs) offer for generative AI developments. Parallelism enables transformers to scale massively and handle complex NLP tasks by building larger models.

Long Short-Term Memory (LSTM)

- Long Short-Term Memory Networks or LSTM in deep learning, is a sequential neural network that allows information to persist. It is a special type of Recurrent Neural Network which is capable of handling the vanishing gradient problem faced by RNN. LSTM was designed by Hochreiter and Schmidhuber that resolves the problem caused by traditional rnns and machine learning algorithms.

- Let’s say while watching a video, you remember the previous scene, or while reading a book, you know what happened in the earlier chapter. RNNs work similarly; they remember the previous information and use it for processing the current input. The shortcoming of RNN is they cannot remember long-term dependencies due to vanishing gradient. LSTMs are explicitly designed to avoid long-term dependency problems.

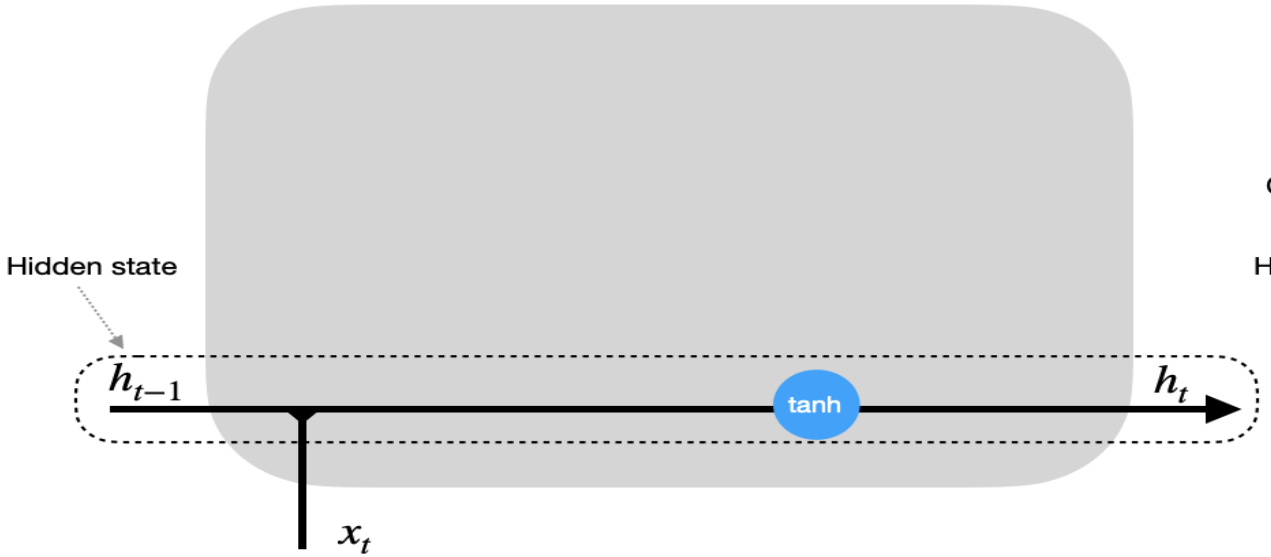
What is LSTM?

- LSTM (Long Short-Term Memory) is a recurrent neural network (RNN) architecture widely used in Deep Learning. It excels at capturing long-term dependencies, making it ideal for sequence prediction tasks.
- Unlike traditional neural networks, LSTM incorporates feedback connections, allowing it to process entire sequences of data, not just individual data points. This makes it highly effective in understanding and predicting patterns in sequential data like time series, text, and speech.

Link to refer

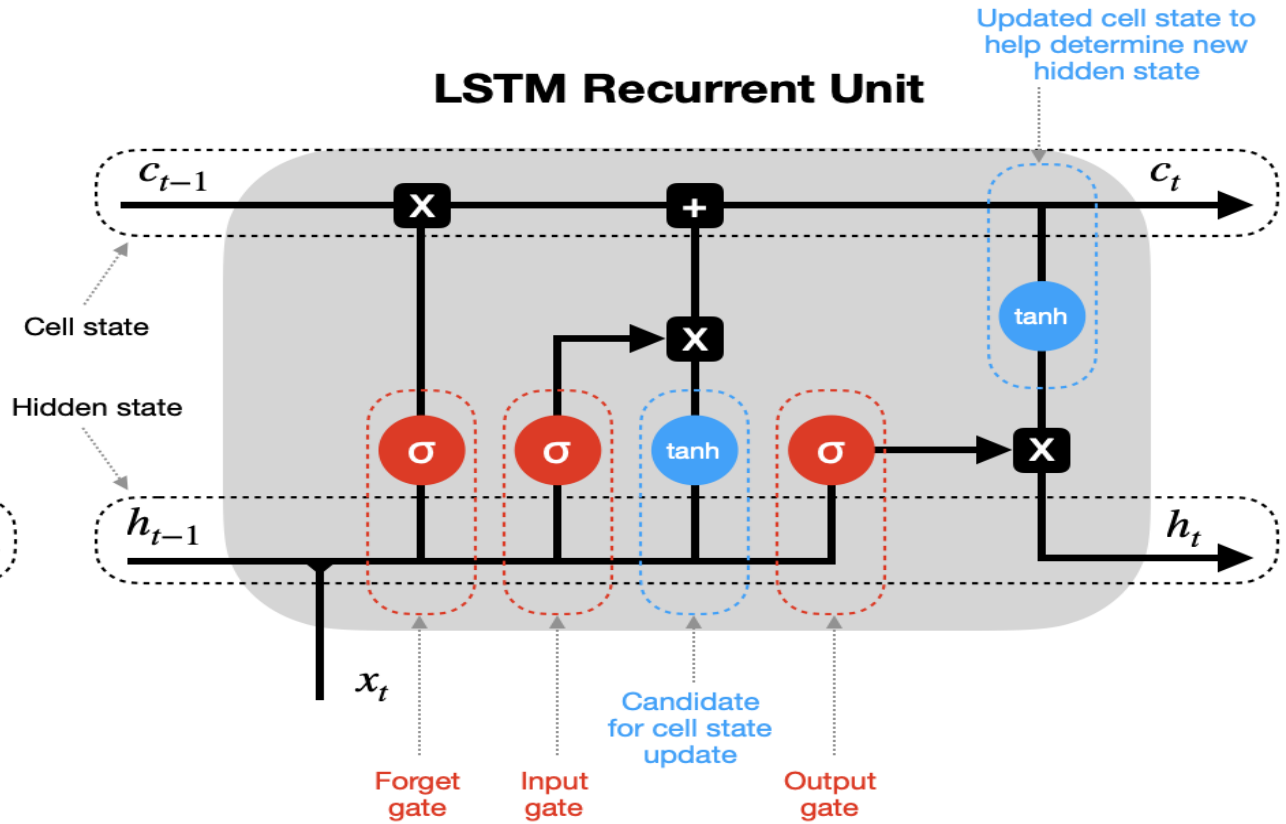
- https://colah.github.io/posts/2015-08-Understanding-LSTMs/

Standard Recurrent Unit



h_{t-1} - hidden state at previous timestep t-1 (memory)
x_t - input vector at current timestep t
h_t - hidden state at current timestep t
tanh - tanh activation function
- concatenation of vectors

LSTM Recurrent Unit

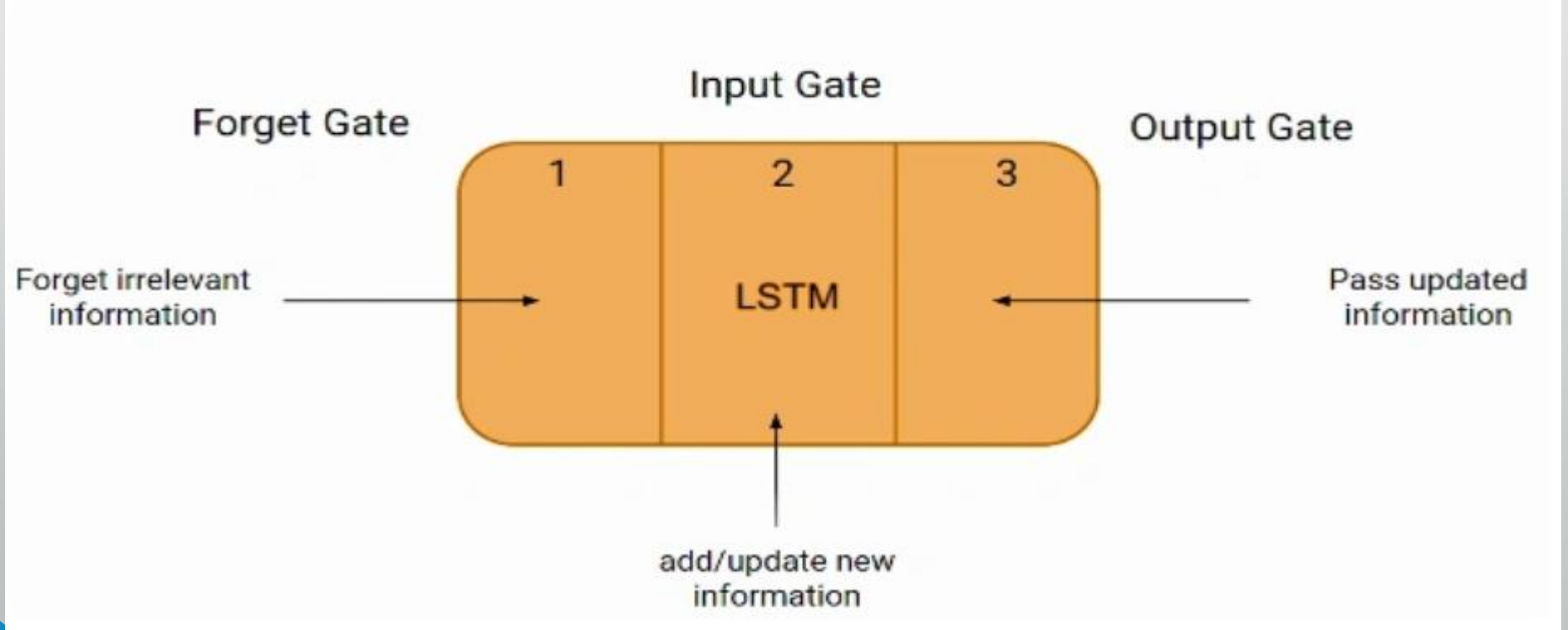


h_{t-1} - hidden state at previous timestep t-1 (short-term memory)
c_{t-1} - cell state at previous timestep t-1 (long-term memory)
x_t - input vector at current timestep t
h_t - hidden state at current timestep t
c_t - cell state at current timestep t

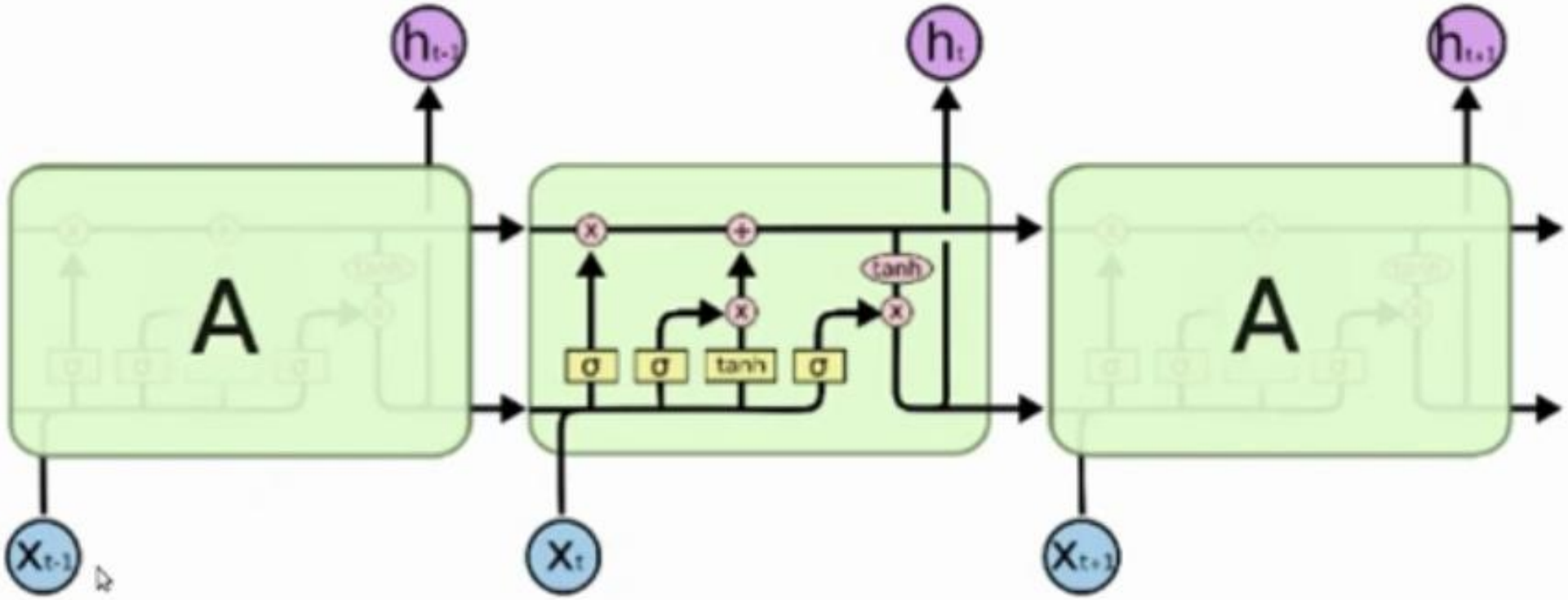
X - vector pointwise multiplication
\+ - vector pointwise addition
tanh - tanh activation function
σ - sigmoid activation function
- concatenation of vectors
- states
- gates
- updates

LSTM Architecture

- The LSTM network architecture consists of three parts:-

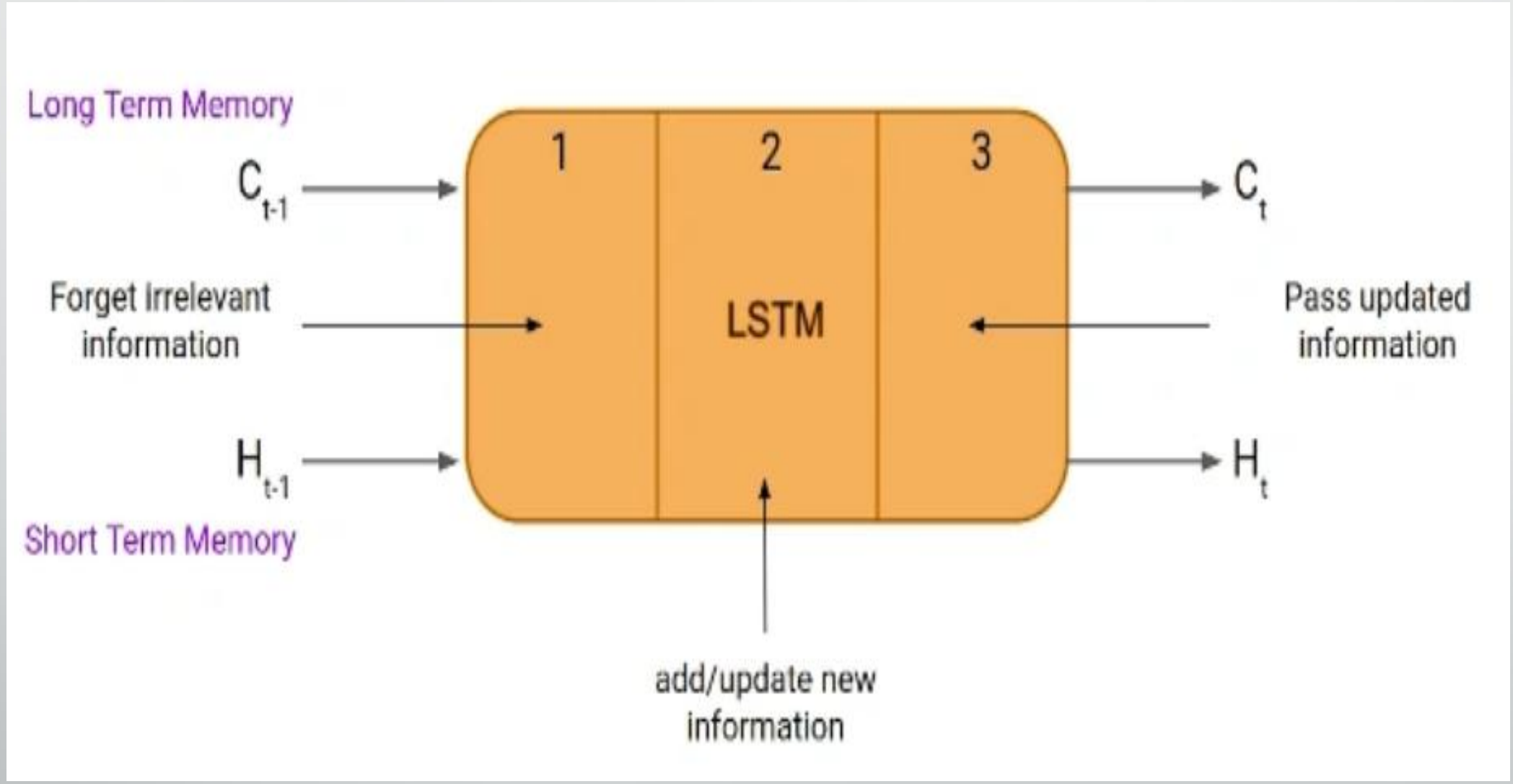


Architecture



- The first part chooses whether the information coming from the previous timestamp is to be remembered or is irrelevant and can be forgotten. In the second part, the cell tries to learn new information from the input to this cell. At last, in the third part, the cell passes the updated information from the current timestamp to the next timestamp. This one cycle of LSTM is considered a single-time step.
- These three parts of an LSTM unit are known as gates. They control the flow of information in and out of the memory cell or lstm cell. The first gate is called Forget gate, the second gate is known as the Input gate, and the last one is the Output gate. An LSTM unit that consists of these three gates and a memory cell or lstm cell can be considered as a layer of neurons in traditional feedforward neural network, with each neuron having a hidden layer and a current state.

- Just like a simple RNN, an LSTM also has a hidden state where H(t-1) represents the hidden state of the previous timestamp and Ht is the hidden state of the current timestamp. In addition to that, LSTM also has a cell state represented by C(t-1) and C(t) for the previous and current timestamps, respectively.
- Here the hidden state is known as Short term memory, and the cell state is known as Long term memory.


Long Term Memory
C_{t-1}
Forget irrelevant information
H_{t-1}
Short Term Memory
1
2
3
LSTM
C_t
Pass updated information
H_t
add/update new information

LSTM

Bob is a nice person. Dan on the other hand is evil.

Example of LTSM Working

- Let's take an example to understand how LSTM works. Here we have two sentences separated by a full stop. The first sentence is "Bob is a nice person," and the second sentence is "Dan, on the Other hand, is evil". It is very clear, in the first sentence, we are talking about Bob, and as soon as we encounter the full stop(.), we started talking about Dan.
- As we move from the first sentence to the second sentence, our network should realize that we are no more talking about Bob. Now our subject is Dan. Here, the Forget gate of the network allows it to forget about it. Let's understand the roles played by these gates in LSTM architecture.

Forget Gate

- In a cell of the LSTM neural network, the first step is to decide whether we should keep the information from the previous time step or forget it. Here is the equation for forget gate.

Forget Gate:

- $f_t = \sigma(x_t * U_f + H_{t-1} * W_f)$

- Let's try to understand the equation, here
- Xt: input to the current timestamp.
- Uf: weight associated with the input
- Ht-1: The hidden state of the previous timestamp
- Wf: It is the weight matrix associated with the hidden state
- Later, a sigmoid function is applied to it. That will make ft a number between 0 and 1. This ft is later multiplied with the cell state of the previous timestamp, as shown below.

$C_{t-1} * f_t = 0$ ...if $f_t = 0$ (forget everything)

$C_{t-1} * f_t = C_{t-1}$ ...if $f_t = 1$ (forget nothing)

Input Gate

- Let's take another example.
- "Bob knows swimming. He told me over the phone that he had served the navy for four long years."
- So, in both these sentences, we are talking about Bob. However, both give different kinds of information about Bob. In the first sentence, we get the information that he knows swimming. Whereas the second sentence tells, he uses the phone and served in the navy for four years.
- Now just think about it, based on the context given in the first sentence, which information in the second sentence is critical? First, he used the phone to tell, or he served in the navy. In this context, it doesn't matter whether he used the phone or any other medium of communication to pass on the information. The fact that he was in the navy is important information, and this is something we want our model to remember for future computation. This is the task of the Input gate.

- The input gate is used to quantify the importance of the new information carried by the input. Here is the equation of the input gate

Input Gate:

- $i_t = \sigma(x_t * U_i + H_{t-1} * W_i)$

Here,

•Xt: Input at the current timestamp t

•Ui: weight matrix of input

•Ht-1: A hidden state at the previous timestamp

•Wi: Weight matrix of input associated with hidden state

Again we have applied the sigmoid function over it. As a result, the value of I at timestamp t will be between 0 and 1.

New Information

$$N_t = \tanh(x_t * U_c + H_{t-1} * W_c) \text{ (new information)}$$

Now the new information that needed to be passed to the cell state is a function of a hidden state at the previous timestamp t-1 and input x at timestamp t. The activation function here is tanh. Due to the tanh function, the value of new information will be between -1 and 1. If the value of Nt is negative, the information is subtracted from the cell state, and if the value is positive, the information is added to the cell state at the current timestamp.

However, the Nt won't be added directly to the cell state. Here comes the updated equation:

$$C_t = f_t * C_{t-1} + i_t * N_t \text{ (updating cell state)}$$

Here, Ct-1 is the cell state at the current timestamp, and the others are the values we have calculated previously.

Output Gate

- Now consider this sentence.
- “Bob single-handedly fought the enemy and died for his country. For his contributions, brave_____.”
- During this task, we have to complete the second sentence. Now, the minute we see the word brave, we know that we are talking about a person. In the sentence, only Bob is brave, we can not say the enemy is brave, or the country is brave. So based on the current expectation, we have to give a relevant word to fill in the blank. That word is our output, and this is the function of our Output gate.

Here is the equation of the Output gate, which is pretty similar to the two previous gates.

Output Gate:

- $o_t = \sigma(x_t * U_o + H_{t-1} * W_o)$

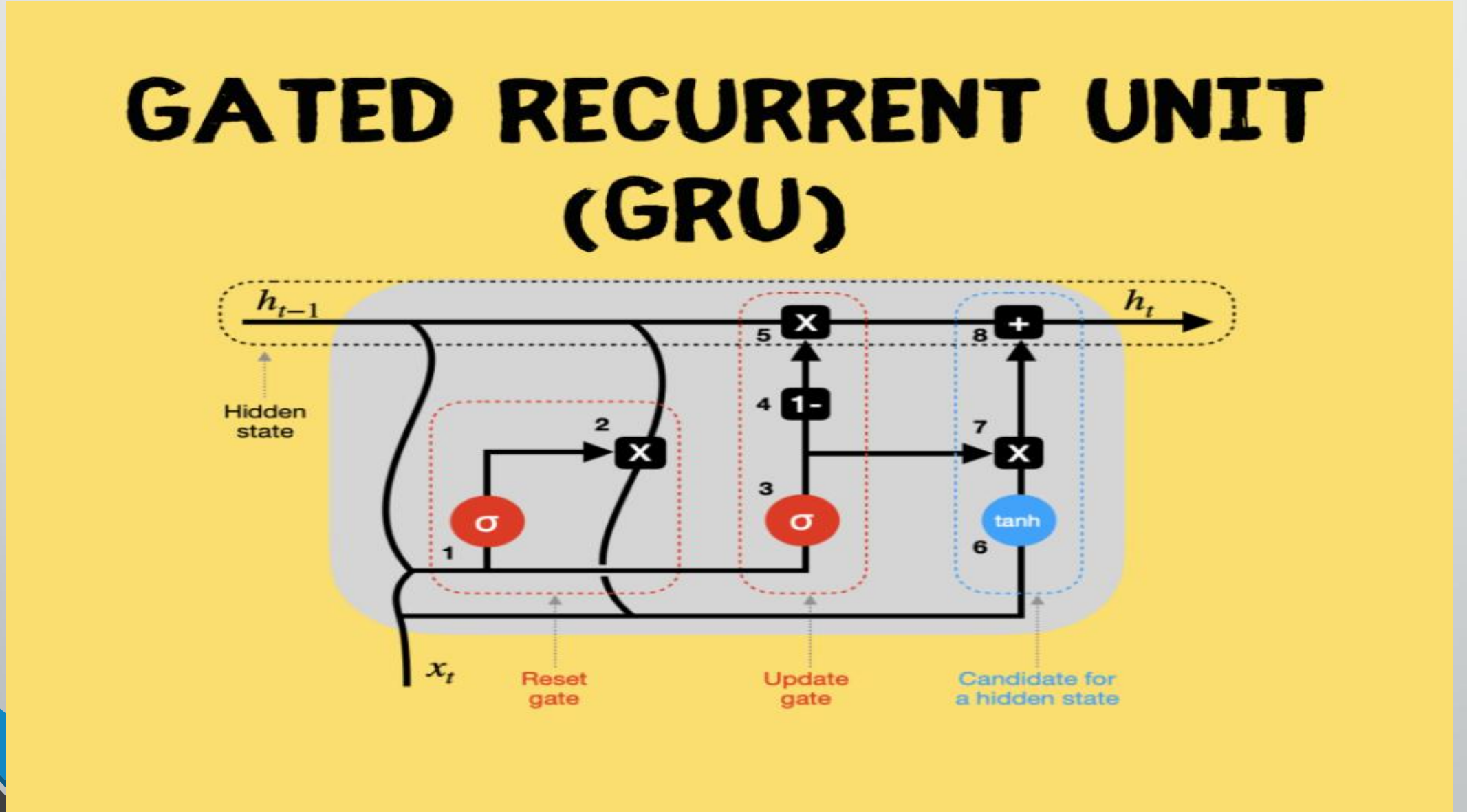
Its value will also lie between 0 and 1 because of this sigmoid function. Now to calculate the current hidden state, we will use Ot and tanh of the updated cell state. As shown below.

$$H_t = o_t * \tanh(C_t)$$

What are Bidirectional LSTMs?

- Bidirectional LSTMs (Long Short-Term Memory) are a type of recurrent neural network (RNN) architecture that processes input data in both forward and backward directions. In a traditional LSTM, the information flows only from past to future, making predictions based on the preceding context. However, in bidirectional LSTMs, the network also considers future context, enabling it to capture dependencies in both directions.
- The bidirectional LSTM comprises two LSTM layers, one processing the input sequence in the forward direction and the other in the backward direction. This allows the network to access information from past and future time steps simultaneously. As a result, bidirectional LSTMs are particularly useful for tasks that require a comprehensive understanding of the input sequence, such as natural language processing tasks like sentiment analysis, machine translation, and named entity recognition.

Gated Recurrent Unit (GRU)



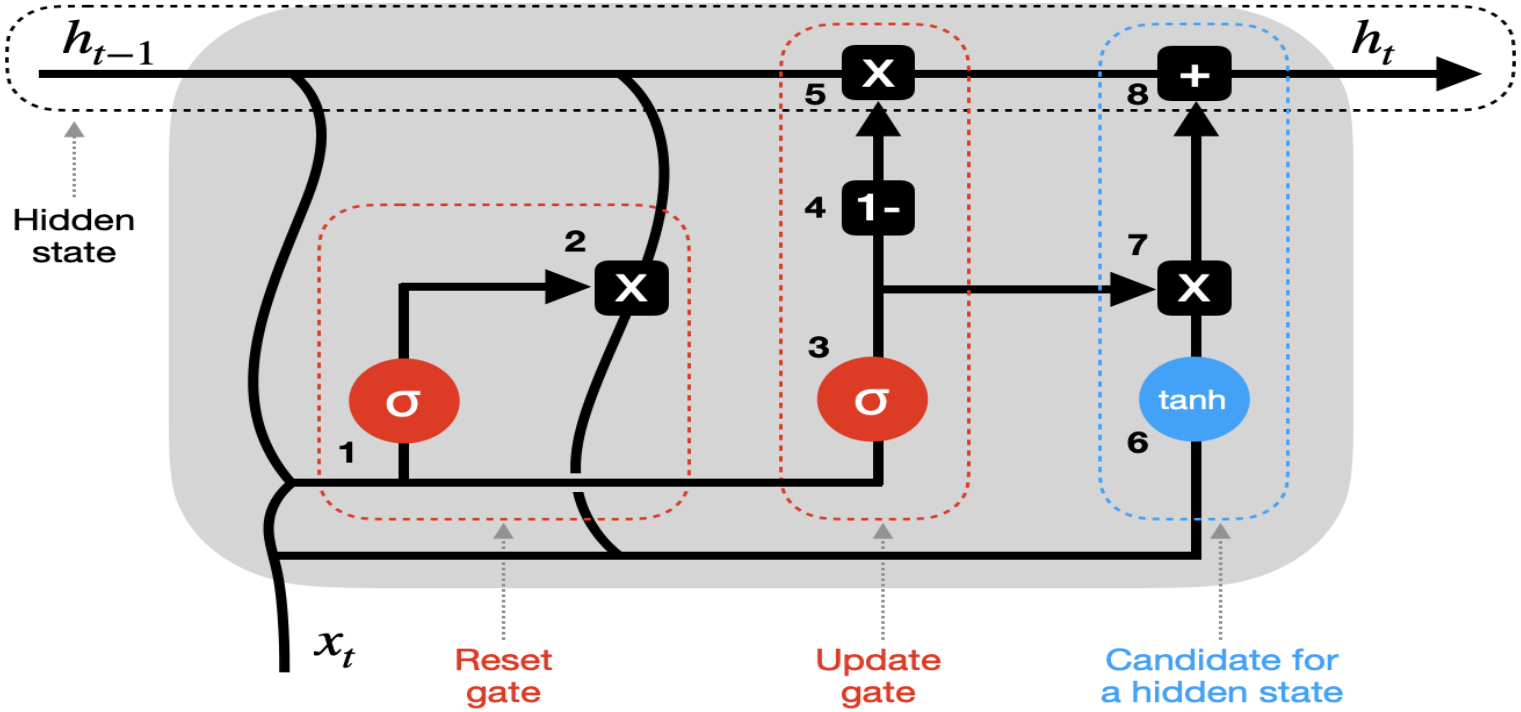
GRU

- **Gated Recurrent Units (GRU)** and **Long Short-Term Memory (LSTM)** have been introduced to tackle the issue of vanishing / exploding gradients in the standard Recurrent Neural Networks (RNNs).

How does GRU work?

- GRU is similar to LSTM, but it has fewer gates. Also, it relies solely on a hidden state for memory transfer between recurrent units, so there is no separate cell state. Let's analyze this simplified GRU diagram in detail (weights and biases not shown).

GRU Recurrent Unit



h_{t-1} - hidden state at previous timestep t-1 (memory)

x_t - input vector at current timestep t

h_t - hidden state at current timestep t

X - vector pointwise multiplication

\+ - vector pointwise addition

tanh - tanh activation function

σ - sigmoid activation function

\- concatenation of vectors

- states

- gates

- updates

- **1–2 Reset gate** – previous hidden state (h_t-1) and current input (x_t) are combined (multiplied by their respective weights and bias added) and passed through a reset gate. Since the sigmoid function ranges between 0 and 1, step one sets which values should be discarded (0), remembered (1), or partially retained (between 0 and 1). Step two resets the previous hidden state multiplying it with outputs from step one.
- **3–4–5 Update gate** – step three may seem analogous to step one, but keep in mind that weights and biases used to scale these vectors are different, providing a different sigmoid output. So, after passing a combined vector through a sigmoid function, we subtract it from a vector containing all 1s (step four) and multiply it by the previous hidden state (step five). That's one part of updating the hidden state with new information.

- **6–7–8 Hidden state candidate** – after resetting a previous hidden state in step two, the outputs are combined with new inputs (x_t), multiplying them by their respective weights and adding biases before passing through a tanh activation function (step six). Then the hidden state candidate is multiplied by the results of an update gate (step seven) and added to previously modified h_t-1 to form the new hidden state h_t.
- Next, the process repeats for timestep t+1, etc., until the recurrent unit processes the entire sequence.

Equations for GRU Operations

- The internal workings of a GRU can be described using following equations:
- **1. Reset gate:**
- $r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$
- The reset gate determines how much of the previous hidden state h_{t-1} should be forgotten.

- **2. Update gate:**
- $z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$
- The update gate controls how much of the new information x_t should be used to update the hidden state.

- **3. Candidate hidden state:**

- ht'=tanh($Wh\cdot[rt\cdot ht-1,xt]$)
- This is the potential new hidden state calculated based on the current input and the previous hidden state.
- **4. Hidden state:**
- $h_t = (1 - z_t) \cdot h_{t-1} + z_t \cdot h'_t$
- The final hidden state is a weighted average of the previous hidden state h_{t-1} and the candidate hidden state h'_t based on the update gate z_t.

How GRUs Solve the Vanishing Gradient Problem

- Like LSTMs, GRUs were designed to address the vanishing gradient problem which is common in traditional RNNs. GRUs help mitigate this issue by using gates that regulate the flow of gradients during training ensuring that important information is preserved and that gradients do not shrink excessively over time. By using these gates, GRUs maintain a balance between remembering important past information and learning new, relevant data.

LSTM vs GRU

Feature	LSTM (Long Short-Term Memory)	GRU (Gated Recurrent Unit)
Gates	3 (Input, Forget, Output)	2 (Update, Reset)
Cell State	Yes it has cell state	No (Hidden state only)
Training Speed	Slower due to complexity	Faster due to simpler architecture

Computational Load	Higher due to more gates and parameters	Lower due to fewer gates and parameters
Performance	Often better in tasks requiring long-term memory	Performs similarly in many tasks with less complexity

TEXT GENERATION

- Text generation is the process of automatically producing coherent and meaningful text, which can be in the form of sentences, paragraphs or even entire documents. It involves various techniques, which can be found under the field such as natural language processing (NLP), machine learning and deep learning algorithms, to analyze input data and generate human-like text. The goal is to create text that is not only grammatically correct but also contextually appropriate and engaging for the intended audience.

Text generation techniques

- **Statistical models:** These models typically use a large dataset of text to learn the patterns and structures of human language, and then use this knowledge to generate new text. Statistical models can be effective at generating text that is similar to the training data, but they can struggle to generate text that is both creative and diverse. N-gram models and conditional random fields (CRF) are popular statistical models.
- **N-gram models:** These are a type of statistical model that uses the n-gram language model, which predicts the probability of a sequence of "n-items" in a given context.
- **Conditional random fields (CRFs):** These are a type of statistical model that uses a probabilistic graphical model to model the dependencies between words in a sentence. CRFs can be effective at generating text that is both coherent and contextually appropriate, but this type of text generation model can be computationally expensive to train and might not perform well on tasks that require a high degree of creative language generation.

- Neural networks: These are machine learning algorithms that use artificial neural networks to identify data patterns. Through APIs, developers can tap into pretrained models for creative and diverse text generation, closely mirroring the training data's complexity. The quality of the generated text heavily relies on the training data. However, these networks demand significant computational resources and extensive data for optimal performance.

- Recurrent neural networks (RNNs): These are a foundational type of neural network optimized for processing sequential data, such as word sequences in sentences or paragraphs. They excel in tasks that require understanding sequences, making them useful in the early stages of developing large language models(LLMs). However, RNNs face challenges with long-term dependencies across extended texts, a limitation stemming from their sequential processing nature. As information progresses through the network, early input influence diminishes, leading to the "vanishing gradient" problem during backpropagation, where updates shrink and hinder the model's ability to maintain long-sequence connections. Incorporating techniques from reinforcement learning can offer strategies to mitigate these issues, providing alternative learning paradigms to strengthen sequence memory and decision-making processes in these networks.
- **Long short-term memory networks (LSTMs):** This is a type of neural network that uses a memory cell to store and access information over long periods of time. LSTMs can be effective at handling long-term dependencies, such as the relationships between sentences in a document, and can generate text that is both coherent and contextually appropriate

- Transformer-based models: These models are a type of neural network that uses self-attention mechanisms to process sequential data. Transformer-based models can be effective at generating text that is both creative and diverse, as they can learn complex patterns and structures in the training data and generate new text that is similar to the training data. Unlike historical approaches such as RNNs and LSTMs, transformer-based models have the distinct advantage of processing data in parallel, rather than sequentially. This allows for more efficient handling of long-term dependencies across large datasets, making these models especially powerful for natural language processing applications such as machine translation and text summarization.

- **Generative pretrained transformer (GPT):** GPT is a transformer-based model that is trained on a large dataset of text to generate human-like text. GPT can be effective at generating text that is both creative and diverse, as it can learn complex patterns and structures in the training data and generate new text that is similar to the training data.
- **Bidirectional encoder representations from transformers (BERT):** BERT is a transformer-based model that is trained on a large dataset of text to generate bidirectional representations of words. That means it evaluates the context of words from both before and after a sentence. This comprehensive context awareness allows BERT to achieve a nuanced understanding of language nuances, resulting in highly accurate and coherent text generation. This bidirectional approach is a key distinction that enhances BERT's performance in applications requiring deep language comprehension, such as question answering and named entity recognition (NER), by providing a fuller context compared to unidirectional models.

Benefits of text generation

- **Improved efficiency:** Text generation can significantly reduce the time and effort required to produce large volumes of text. For instance, it can be used to automate the creation of product descriptions, social media posts or technical documentation. This not only saves time but also allows teams to focus on more strategic tasks.[2]
- **Enhanced creativity:** Artificial intelligence can generate unique and original content at high speed, which might not be possible for humans to produce manually. This can lead to more innovative and engaging content, such as stories, poems or music notes. Also, text generation can help overcome writer's block by providing new ideas and perspectives.
- **Increased accessibility:** Text generation can assist individuals with disabilities or language barriers by generating text in alternative formats or languages. This can help make information more accessible to a wider range of people, including those who are deaf or hard of hearing, non-native speakers or visually impaired.

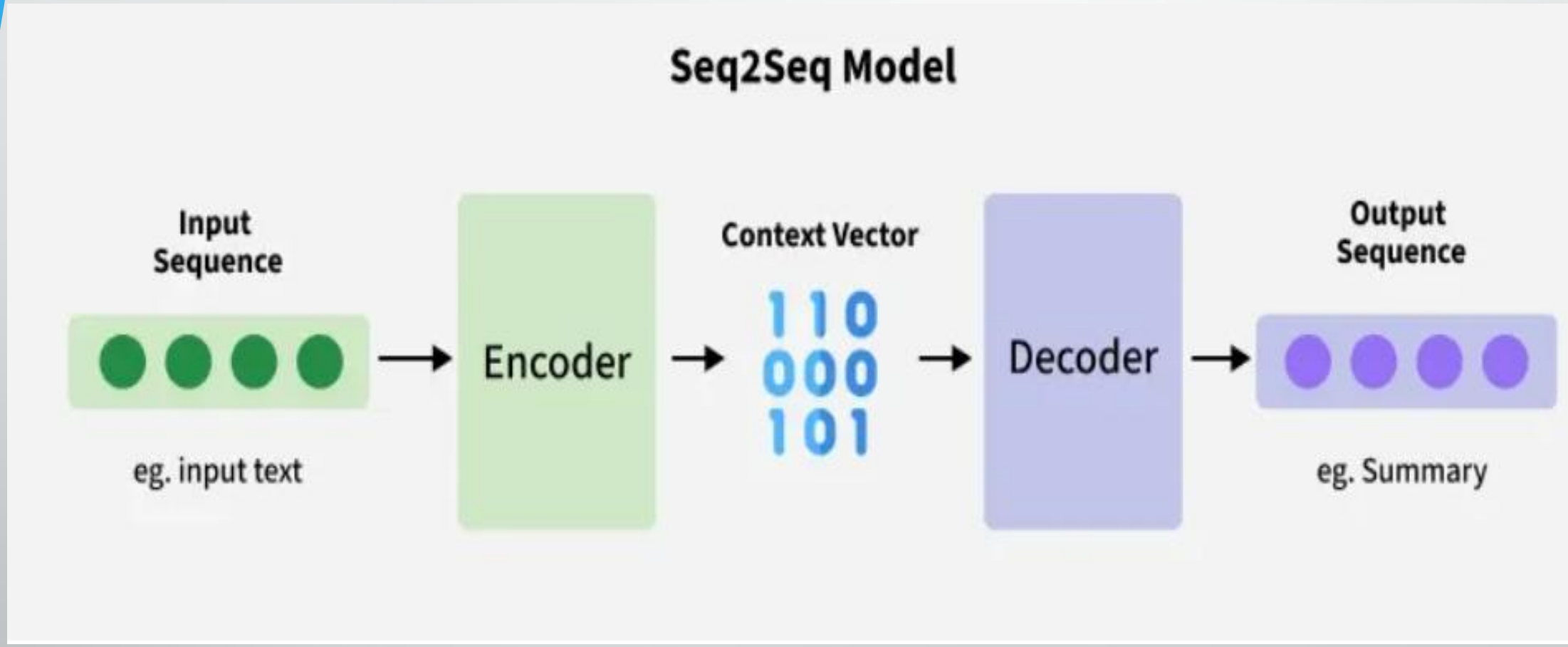
- **Better customer engagement:** Personalized and customized text generation can help businesses and organizations better engage with their customers. By tailoring content to individual preferences and behaviors, companies can create more meaningful and relevant interactions, leading to increased customer satisfaction and loyalty.
- **Enhanced language learning:** Text generation can be a useful tool for language learners by providing feedback and suggestions for improvement. By generating text in a specific language style or genre, learners can practice and develop their writing skills in a more structured and guided way.

Challenges of text generation techniques

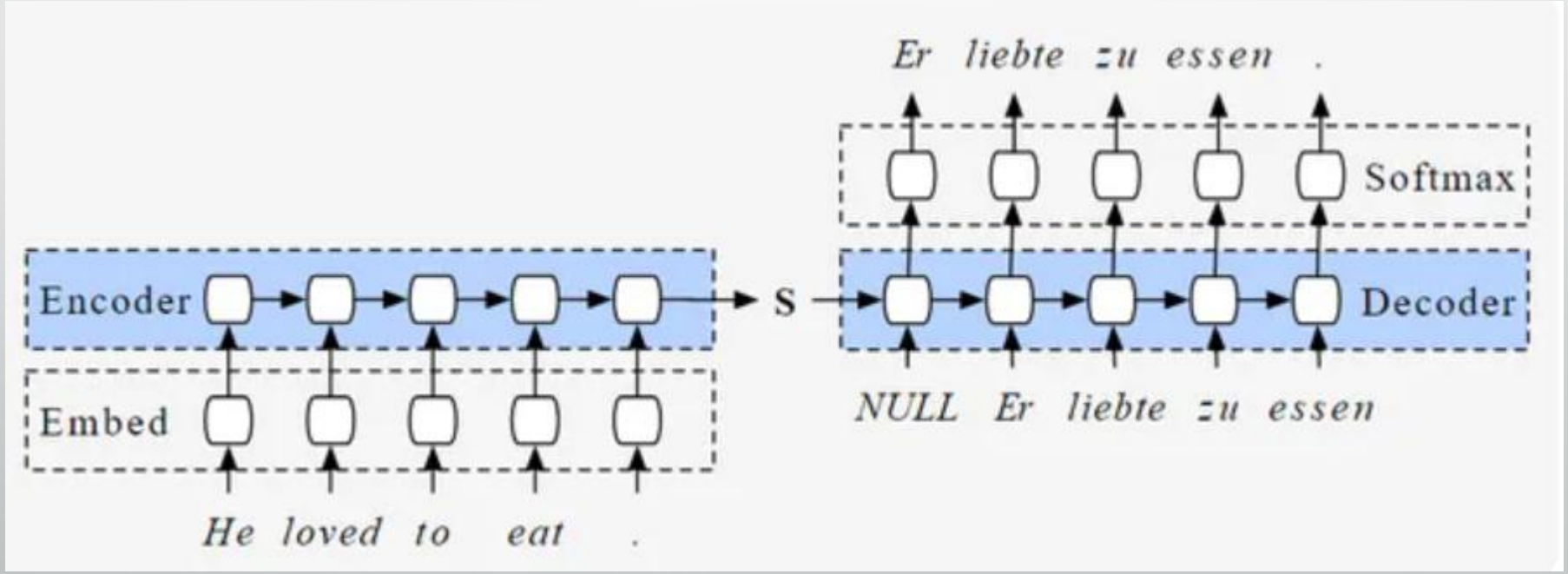
- **Quality:** One of the most significant challenges in text generation is ensuring the quality of the generated text. The generated text should be coherent, meaningful and contextually appropriate. It should also accurately reflect the intended meaning and avoid generating misleading or incorrect information.
- **Diversity:** A second challenge in text generation is promoting diversity in the generated output. While it is important for the generated text to be accurate and consistent, it is also crucial that it reflects a wide range of perspectives, styles and voices. This challenge is particularly relevant in applications such as natural language processing, where the goal is to create text that is not only accurate but also engaging and readable.
- **Ethics and privacy:** A third challenge in text generation is addressing ethical considerations and privacy concerns. As text generation techniques become more sophisticated, there is a risk that they might be used to generate misleading or harmful text or to invade people's privacy.

Language Translation
1. Seq to Seq Model

- **Encoder:**
- **Input Processing:** The encoder takes the source language sentence as input. This sentence is typically broken down into a sequence of words or tokens.
- **Encoding Step-by-Step:** The encoder processes each word in the sequence one at a time. It often uses Recurrent Neural Networks (RNNs), particularly LSTMs (Long Short-Term Memory), to handle long sentences effectively. The RNN considers the current word and the information accumulated from previous words at each step.
- **Context Vector Generation:** The encoder's goal is to compress the meaning of the entire source sentence into a single vector, called the context vector. This vector encapsulates the vital information from the sentence, including its meaning, structure, and relationships between words.


Seq2Seq Model
Input Sequence
eg. input text
Encoder
Context Vector
1 1 0
0 0 0
1 0 1
Decoder
Output Sequence
eg. Summary

The entire process can be visualized as a bridge. The encoder takes the source language sentence and builds a bridge (context vector) representing its meaning. The decoder then uses this bridge to walk across, generating the target language sentence word by word.



- # Decoder:
- 1. **Initialization:** The decoder takes the context vector generated by the encoder as its starting point. This vector serves as a condensed representation of the source language sentence.
- 2. **Output Generation Step-by-Step:** The decoder uses an RNN (often an LSTM) to generate the target sentence word by word. At each step, the decoder considers two things:
- The context vector from the encoder provides the overall meaning of the source sentence.
- The previously generated word(s) in the target language sequence allow the decoder to build the target sentence coherently.
- 3. **Probability Prediction:** For each step, the decoder predicts the probability of the next word in the target language sequence. This prediction is based on the information received from the context vector and the previously generated words.
- 4. **Target Sentence Construction:** The decoder iterates one word at a time through these steps until the target language sentence is complete. The most likely word at each step is chosen to build the final translated sentence.

- Utilization of Context Vector in Decoder
- The decoder in a seq2seq model plays a critical role in translating the encoded meaning of the source language into a fluent target language sentence. It achieves this by cleverly utilizing two sources of information at each step of the translation process:
- **Context Vector:** This vector, generated by the encoder, acts as a compressed representation of the entire source sentence. It captures the essential meaning, structure, and relationships between words. The decoder attends to this context vector throughout the translation process, ensuring the generated target language sentence reflects the original meaning.
- **Internal State:** The decoder, often a recurrent neural network (RNN) like LSTM, maintains an internal state. This state acts like a memory, keeping track of the previously generated words in the target language sequence. This information is crucial for generating grammatically correct and coherent sentences.

- How do these two elements work together?
- **Initial Step:** At the beginning, the decoder receives the context vector from the encoder. This vector provides a high-level understanding of the entire source sentence.
- **Word Prediction:** For each target word, the decoder uses both the context vector and its internal state to predict the most likely next word in the target sequence. This prediction considers:

  - **Relevance to Context:** The decoder checks the context vector to ensure the predicted word aligns with the overall meaning of the source sentence.
  - **Grammatical Consistency:** The decoder uses its internal state, which holds information about previously generated words, to predict a word that makes grammatical sense in the current context of the target sentence.

- **Internal State Update:** After predicting a word, the decoder updates its internal state. This update incorporates the newly generated word, allowing the decoder to remember the evolving target language sequence.
- **Iterative Process:** The decoder continues this process of using the context vector and its internal state to predict the next word, one at a time, until the entire target language sentence is generated.

- By effectively combining the information from the context vector and its internal state, the decoder can:
- **Maintain Coherence:** It ensures the generated target language sentence flows smoothly and logically, reflecting the original meaning.
- **Capture Grammar and Syntax:** It leverages information about previously generated words to construct grammatically correct sentences in the target language.
- Overall, the interplay between the context vector and the decoder's internal state is what allows seq2seq models to translate languages in a way that is both accurate and fluent.

- RNNs and LSTMs in Seq2Seq Models
- Seq2seq models rely on Recurrent Neural Networks (RNNs) as their core building block to handle the sequential nature of text data. RNNs are a special kind of neural network designed to process sequences like sentences.
- Here's how RNNs capture sequential information:
- **Internal State:** Unlike traditional neural networks, RNNs have an internal state. This state acts like a memory, allowing the network to consider not just the current input but also the information from previous inputs in the sequence.
- **Sequential Processing:** RNNs process information step-by-step. At each step, they take the current input and combine it with their internal state to generate an output and update their internal state for the next step. This way, information from previous elements in the sequence can influence the processing of later elements.

- In seq2seq models, LSTMs are often used in both the encoder and decoder. The encoder uses LSTMs to process the source language sentence and capture its meaning in the context vector. The decoder then leverages LSTMs to generate the target language sentence word by word, considering both the context vector and the previously generated words in the target sequence. This allows seq2seq models to effectively translate languages even for longer sentences.

Language translation through attention mechanism

- **Refer below link**
- https://www.geeksforgeeks.org/artificial-intelligence/ml-attention-mechanism/